When a Copy Is Not a Continuation

Ioannis Tsiokos

Preprint v1.0 14 October 2026

Abstract

Thought experiments about copying, teleporting and uploading ask whether the result is the same subject. We ask a prior question that can be answered exactly. When the organization described by a closure architecture for consciousness is moved to a different finite system, what would make the result a continuation rather than a copy? We show that the question splits into three separately checked parts: whether the target realizes the architecture's conditions, whether the episode passes through handoffs in which both systems are live with no gap, and whether the target keeps itself going once the source and every transfer aid are withdrawn. A certificate that checks the three is sound, finite models pass it, and finite countermodels show what fails without each part. A detached copy and a gap fail the lineage check, and success up to any finite horizon does not establish unbounded maintenance. A public shadow shows that public observations lose information about the predictive state. Under symmetric fission, continuation is a relation with two values, and nothing in the structure picks one continuer. On one designed model, all eight conditions, as adopted at interfaces fixed in advance, are computed jointly from one execution, and a weaker continuation, by ancestry, holds for both branches of a fission and for a guarded fusion once each has completed its own acquisition. Every result is checked in Lean with standard axioms only. We claim nothing about consciousness, survival, numerical identity or substrate independence.

Keywords. continuation, personal identity, fission, viability, finite models, Lean, mathematical consciousness science.

1 Introduction

Philosophers have long asked what happens to a subject when its organization is moved. A teleporter scans a person and builds an exact replica elsewhere. A gradual procedure replaces a brain part by part. An upload copies the pattern and leaves the original running (Parfit, 1984; Chalmers, 2010; Cerullo, 2015). The usual question is whether the result is the same subject. That question has resisted settlement, and we do not try to settle it here. We ask a prior question that can be answered exactly. Suppose the organization described by a closure architecture for consciousness (Tsiokos, 2026j) is moved from one finite system to a different one. What, precisely, would make the result a *continuation* of the source rather than a *copy* of it, and which of the conditions involved can be checked?

We show that the question splits into three, and that the three must be checked separately. The first is *realization*: does the target system satisfy the conditions of the architecture, on its own state space? The second is *lineage*: does the episode pass from source to target through handoffs in which the two systems are live at the same time, with no gap? The third is *autonomy*: once the source and every aid used for the transfer are withdrawn, does the target keep itself going? A detached copy can pass the first and the third and fail the second. A resumed stream can match the source

Automorph Inc., Wilmington, DE, USA ioannis@automorph.io
DOI: 10.5281/zenodo.23233188 © 2026 CC-BY 4.0 Preprint, not peer reviewed

at both ends and still fail the second. A target that runs only while an outside resource is supplied fails the third. Similarity of endpoints settles none of the three.

Our main result is a precise form of this separation. We define three checks whose predicates are defined from the systems' own data. Realization and lineage are computed from it. Autonomy additionally requires a supplied proof of an invariance obligation. We prove that a certificate passing them is sound: it yields realization on the target, a strict live-overlap lineage, and autonomy after withdrawal (Theorem 3, the continuation theorem). The proofs are elementary. Local pieces that agree where they overlap glue into a single realization on the target. A one-step agreement between source and target extends to equality of their observation laws over every finite sequence of actions. A certificate of invariance yields a strategy that keeps the target going. The work lies elsewhere: in formulating checks strong enough to separate continuation from its look-alikes, and in exhibiting exact finite systems that pass them and others that fail. The theorem comes in two versions. In the operational version, each of the three conclusions can fail while the other two hold. In the realization-preserving version, autonomy is defined to include realization, so it implies it. Our passing examples include a chain of handoffs through three distinct systems. On these models four of the eight conditions hold only in a simplified form, which we describe.

The reason endpoint similarity cannot decide the matter is short, and we give it informally first.

(P1) A copy can be built on a different system so that its final predictive state matches the source's.

(P2) The copy can satisfy the conditions on its own system and keep itself going without help.

(P3) If there is no stage at which source and copy are live together, no handoff carries the source's ongoing organization into the copy.

(C) So realization and autonomy can both hold, with matching endpoints, while lineage fails. Any criterion that looks only at endpoints, or only at the target, cannot tell this copy from a continuation.

Section 3 turns this argument into a finite countermodel, in which the conditions hold in the simplified form described there, and shows two further ways to match endpoints without continuing: a gap in which the stream stops and later resumes, and a re-encoding of one system that never changes system at all.

Three further results complete the picture. First, continuation is a relation, not a function. When one source continues symmetrically into two targets, no rule that respects the symmetry picks exactly one of them as the continuer, and ancestry alone does not settle which, if either, is the original (Section 7). Second, what a target inherits depends on the route the transfer took. Two histories can end in the same current state and still differ in what they predict, so the target's present must carry enough to tell such histories apart. Keeping route memory is one way to do this, and a flatness or confluence argument is a sufficient, not a necessary, ground for dropping it. Holonomy can rule out a present that is fixed by the shared boundary alone (Section 6). Third, we exhibit one designed model, with a finite physical state space and unbounded execution histories, on which all eight conditions of the architecture, as adopted at interfaces fixed in advance, are computed jointly from a single execution. On this model a weaker form of continuation holds, continuation by ancestry, for both branches of a fission and for the output of a guarded fusion, each once it has completed its own acquisition (Section 8). Its lineage is predictive ancestry, which is weaker than live overlap. This is not a certificate for the full architecture, whose complete panel of checks we do not verify.

What we establish, and what we do not, can be said once. Every result is a statement about finite systems, some with unbounded histories, and every result is checked in the Lean theorem prover (de Moura and Ullrich, 2021) using its core library and standard axioms only. The results

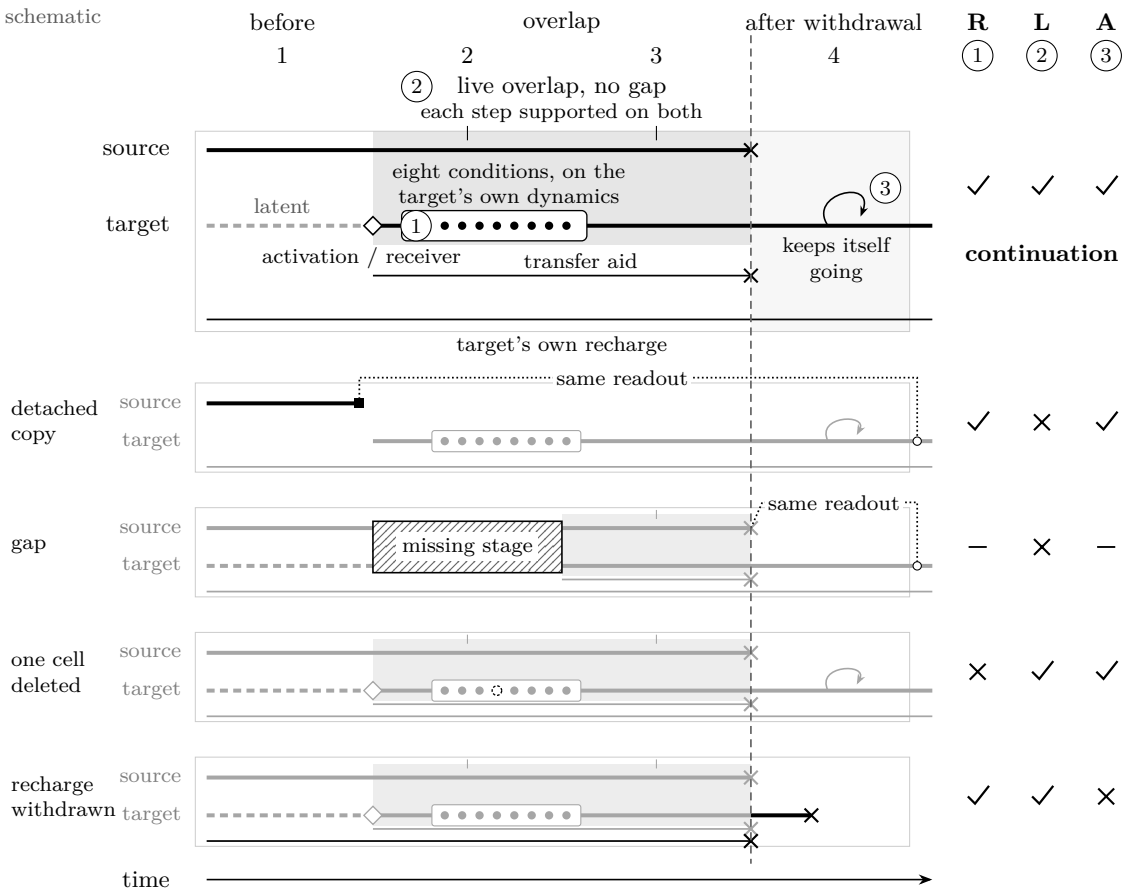


Figure 1. One handoff and four look-alikes. The top row is a handoff that passes all three checks. Realization is checked on the target's own dynamics. Lineage is checked where source and target are live together, with no gap. Autonomy is checked after the source and every transfer aid are withdrawn. The rows below are separate proved controls. A detached copy and a gap have the same predictive readout as the source at the endpoint, and both fail lineage. The last two rows each change one thing in the passing handoff. Deleting one published cell fails only realization. Withdrawing the target's retained recharge fails only autonomy. A dash marks a check we do not evaluate for that case. Schematic. The rows follow the operational version, in which none of the three checks implies another. On the proved instances, conditions (4) to (7) hold in simplified form.

are structural. They do not say that any system is conscious, that consciousness can or cannot be moved between substrates, or that a continuation is the same person or subject. The architecture's postulate of Substrate Indifference remains a postulate, and our results about moving structure between finite systems do not address it. Nothing here shows that any measurement of a real system identifies these structures. Any step from our results to felt continuity needs hypotheses that we state and do not assume. Finally, no single model in this paper has both strict live-overlap lineage and all eight conditions at the grade the architecture specifies. Exhibiting one is an open problem.

Our contributions are four.

1. We state realization, lineage and autonomy as separate predicates on finite systems, and prove that a certificate checking them is sound (Sections 4 and 5).

2. We give finite countermodels for the shortcuts: a detached copy, a gap, a public shadow, agreement checked only between neighbouring pieces, a clock that ticks without progress, a summed defect score, and success up to any finite horizon (Sections 3 and 4).
3. We show that continuation is relation-valued under fission, that no symmetry-respecting rule selects a unique continuer, and that ancestry alone underdetermines identity (Section 7).
4. We exhibit a model on which all eight conditions are computed jointly, and prove continuation by ancestry on it, including guarded fusion (Section 8).

Section 2 fixes the setting and says how to read our claims. Section 3 works through the copy example. Section 4 gives the three checks, and Section 5 the continuation theorem. Sections 6 and 7 treat routes and branching, Section 8 continuation by ancestry, and Section 9 direct succession and the reach of the results. Section 10 answers objections, Section 11 places the work, and Section 12 concludes.

2 Setting, and how to read our claims

2.1 Finite systems with unbounded histories

Every system we study has a finite physical state space, and finite sets of actions, disturbances and outcomes. Runs, histories and event registers may be unbounded, and so may the observations that report them. The temporal fields include counters of order, progress and records, which grow without bound. Where they are, our results about them are proved by invariants and induction, not by listing cases. When a result concerns moving structure from one system to another, the source and the target are distinct systems, fixed before anything is evaluated. Re-encoding a single system does not count, and we show in Section 3 why the distinction matters.

2.2 The eight conditions

We take the conditions from the closure architecture of Tsiokos (2026j) and recall them only briefly. A system meets the architecture when it satisfies the following eight conditions jointly, on one system, with data tying them together:

1. *objecthood*: staged closure kernels have robust fixed points
2. *strict refinement*: some finer description is not recoverable from the public one
3. a *predictive self-coordinate*: refining the public description by the system's own dynamics stabilizes at a predictive quotient
4. *non-exposure*: the predictive state is not exposed at the given public channels
5. *action credit*: directionality is credited to self, other or none, by where it is generated
6. *temporal closure*: a funded order, tick and arrow carried by the system's own record
7. *binding*: joint content over branches strictly refines the branch-by-branch description
8. *viability*: a nonempty viability kernel under maintenance, which underfunding empties.

Each of our models meets these conditions at a recorded grade.

- A condition holds at *design grade* when it is computed in the form the architecture specifies, at an interface fixed in advance.

Table 1. The grade at which each of the eight conditions holds on each model. "Computed" means computed from the model's own dynamics. "Simplified" means a declared stand-in, described in Section 2. "Adopted interface" means computed jointly from one execution at interfaces fixed in advance, with the noted choices.

Condition	Two-copy model	Chain model	Fusion model
(1) Objecthood	computed	computed	adopted interface[a]
(2) Strict refinement	computed	computed	adopted interface
(3) Predictive self-coordinate	computed	computed	adopted interface
(4) Non-exposure	simplified	simplified	adopted interface[b]
(5) Action credit	simplified	simplified	adopted interface[c]
(6) Temporal closure	simplified	simplified	adopted interface[c]
(7) Binding	simplified	simplified	adopted interface
(8) Viability	computed	computed	adopted interface

[a] Physical stationarity excludes growing histories; the period-two two-step counterexample is retained. [b] Current observations on maintained support, not all future observations. [c] Completed local-cycle evidence with continuously authenticated current residency and mandatory current-generator, ownership and control links.

- It holds in *simplified form* when a declared stand-in replaces it.

On the two models that carry our strict continuation results, conditions (4) to (7) hold only in simplified form:

- non-exposure is replaced by a single collision between two states that look the same and predict differently
- action credit by a declared credit rule
- temporal closure by a declared order on events with a record and tick added for the purpose
- binding by a declared coupling.

The first replacement shows why these forms are weaker. A single collision shows that the public channels cannot recover the whole predictive state. Full non-exposure asks for more: that no public channel reveals any feature of the predictive state that varies. On the two-copy model it fails, because the public position is itself such a feature. The collision survives that exposure, and full non-exposure does not.

On the third model, all eight conditions, as adopted at interfaces fixed in advance, are computed jointly from a single execution, with stated choices for conditions (1), (4), (5) and (6) (Section 8). That is not a certificate for the full architecture, whose complete panel of checks we do not verify. Table 1 gives the details. We use the shorter phrase "the target realizes the conditions" only with the grade stated nearby.

Realization in our sense is a check on the target's own states and transitions. It resembles the functionalist sense in which a physical system realizes a functional organization when its components and their dependencies reflect those of the specification (Chalmers, 1995). The observation that one organization can be realized by different systems goes back to Putnam (1967). Whether passing the check is being conscious is not part of the check.

2.3 The temporal signature

What a continuation must carry across, beyond the conditions themselves, is a temporal organization. We fix it in advance as eight fields:

- the predictive present
- an order on events
- a tick phase
- genuine progress
- an arrow given by the growth of the system's own records
- memory of the route by which the present was reached
- a maintenance state
- an owner.

We call these together the *temporal signature*. It is fixed before, and independently of, any judgement about lineage. The arrow in this list is an ordering by record growth. It is not a thermodynamic arrow of time, and none of our results concerns one.

2.4 How to read our claims

Our claims are of four kinds, and a result of one kind never supplies a claim of another.

- *Structural results*, theorems and their finite instances, are proved in Lean.
- *Finite countermodels* refute a stated shortcut on an explicit finite system, also checked in Lean. A countermodel refutes the claim it represents on its stated boundary. It does not refute every way of formulating the shortcut.
- *Interpretive hypotheses* link structure to experience, for example that felt continuity requires the continuation of realized temporal closure. We state them, mark them, and never use them as premises.
- *Ontological postulates*, such as the architecture's realization postulate and Substrate Indifference, are premises the reader may grant or decline. No result here uses them or supports them.

Words like transfer, upload, survival and same person appear only when we describe others' thought experiments or say what our results do not show. They never appear in the conclusion of a result.

Every theorem and countermodel is a declaration in the Lean development that accompanies our results, built against Lean's core library with the standard axioms only. Appendix C maps each result to its declarations. The kernel checks the Lean statements. That those statements say what our prose says is a separate matter. We checked it in a separate review, statement by statement, described under Data and code availability.

3 A copy that is not a continuation

We begin with a worked example. It makes the three questions concrete and shows why endpoint similarity cannot answer them.

Two copies of a deadline system. The companion forcing paper studies a small finite system: a walker on a wheel with a hidden phase and a deadline (Tsiokos, 2026k). Its viable states form a kernel on which a predictive self-coordinate is forced. We take two copies of that system and tag them *source* and *target*. The two tagged copies, together with the deadline system's lethal states, give a finite state space of 24 states, of which 4 are lethal. To this we add the apparatus of a handoff: a position along the episode, a budget of tokens, and a register recording which route the episode took.

A handoff runs through four stages. In the first, only the source is live. In the second and third, both systems are live: the source still acts, the target already acts, and each step is supported on both. In the fourth, the source has been withdrawn and the target continues alone. We call the middle stages the *overlap*. The target's copy of the deadline system is present from the start, but it is latent. It becomes active only at an overlap event, with the source present and a funded receiver. At that event the receiver reads the position and phase at the end of the source's history and writes them into the target. The source is withdrawn only afterwards. This transfer rule is part of the model's design. It does not show the target's organization being built step by step, and it is not independent evidence about how a transfer would have to work.

On this model, conditions (1), (2), (3) and (8) of the architecture hold on each copy in substantive form, computed from the deadline system's own dynamics. Conditions (4) to (7) hold in the simplified form described in Section 2. Everything we say about this model is relative to that grade.

The handoff passes. On this model a single handoff, from the source copy to the target copy, passes all three checks: the target realizes the conditions at the stated grade, the episode is a strict live-overlap lineage, and after the source and the transfer aids are withdrawn the target keeps itself going. This is an instance of the continuation theorem of Section 5, with one difference between how lineage and autonomy draw on the source that we state there. We now vary the picture in four ways. Each variation keeps something that looks like success. Two fail the lineage check, one fails a typing requirement, and one shows that public observations lose information.

The detached copy. Suppose the target's state has the same predictive readout as the source's final state, and the target satisfies the conditions and maintains itself, but at no stage are source and target live together: one stage has only the source, the next only the target. We prove that each of these can hold together with the last (Proposition 4). The endpoint readouts agree, the target realizes the conditions at the stated grade, and it is autonomous. Yet no overlap ever carried the source's running organization into it, and no strict handoff certificate exists for this sequence of stages. The proposition is not deep. The lineage check requires a stage with both systems live, and the point of the example is that the other two checks, and agreement at the endpoint, can all hold without one. Whatever one wants to say about such a copy, it is not a continuation in the sense of our lineage check, and nothing at its endpoints reveals this. Theories that analyse persistence by psychological continuity already treat the cause of a match as a parameter. Parfit's versions of the psychological criterion differ exactly over which causes count (Parfit, 1984, §78). Continuity theorists also disagree over whether copying mental contents supplies the right kind of causal dependence (Olson, 2023, §4; compare Shoemaker and Swinburne, 1984 and Unger, 1990). Our lineage check is not a criterion of persistence. Like those debates, it looks at the route rather than at the endpoints.

The gap. Suppose instead that the episode has the right endpoints, but the stream stops for a while and then resumes. We prove that on the two-copy model, removing one overlap stage from

the realized episode leaves the source and target stages in place, leaves the two routes' endpoints in agreement, and keeps equal predictive readouts at the endpoint, while the no-gap condition of the lineage check fails (Proposition 5). In words, it is a resumed stream rather than a continuation without a gap. A stage is missing from the realized episode, so the no-gap check fails. We do not claim that anything is actively transferred across the gap.

The re-encoding. Suppose the "target" is the source system itself, with its states relabelled by a nontrivial permutation. Such a re-encoding does not meet the requirement that source and target be distinct systems, so it is not transport between systems, however different the relabelled description looks (Remark 1). The point is one of typing, not a theorem.

The public shadow. Finally, suppose we judge the target only by what it shows through its public channel. We prove that two target states can agree on every observation through that fixed channel while their predictive states differ (Proposition 2). Public observations through this interface do not determine the predictive state. Whatever a realization check needs to know about the predictive state, it cannot read it off this interface. This is a statement about the fixed public interface of this finite model. It is not a claim that no observation could ever reveal the predictive state.

What the example shows. Each variation keeps something that a criterion based on endpoints or public behaviour would count as success. The detached copy and the gap each fail the lineage check. The re-encoding fails a typing requirement before any check applies. The public shadow fails no check: it shows that public observations lose information about the predictive state, which the checks use. That is why we state realization, lineage and autonomy as separate predicates, and why the certificate in Section 5 checks them separately. The lower rows of Figure 1 draw the detached copy and the gap beside the passing handoff.

4 Three questions, three checks

In this section we define the three checks and prove what each one delivers. We state the definitions in words here. Appendix B gives them in full for each model, and Appendix C names the Lean declarations.

4.1 Realization on the target

We assemble a realization on the target from local pieces, one for each condition. Each piece reads some of the target's cells: tables of transition weights, positions, records and so on. It must be accepted by its own condition. We ask when accepted pieces fit together into one assignment of values to all the cells.

Definition 1 (Cover with running intersection)**.** A *cover* is a family of pieces, each with a scope (the cells it reads), such that every cell lies in some scope, together with an adjacency relation on pieces. It has the *running-intersection* property if any two pieces whose scopes contain a cell c are joined by a path of adjacent pieces whose scopes all contain c. Local assignments are *compatible along edges* if adjacent pieces assign the same value to every cell they share.

Theorem 1 (Gluing)**.** *Let a rooted cover have the running-intersection property, and let local assignments be compatible along edges. Then there is a global assignment whose restriction to each piece's scope is that piece's local assignment.*

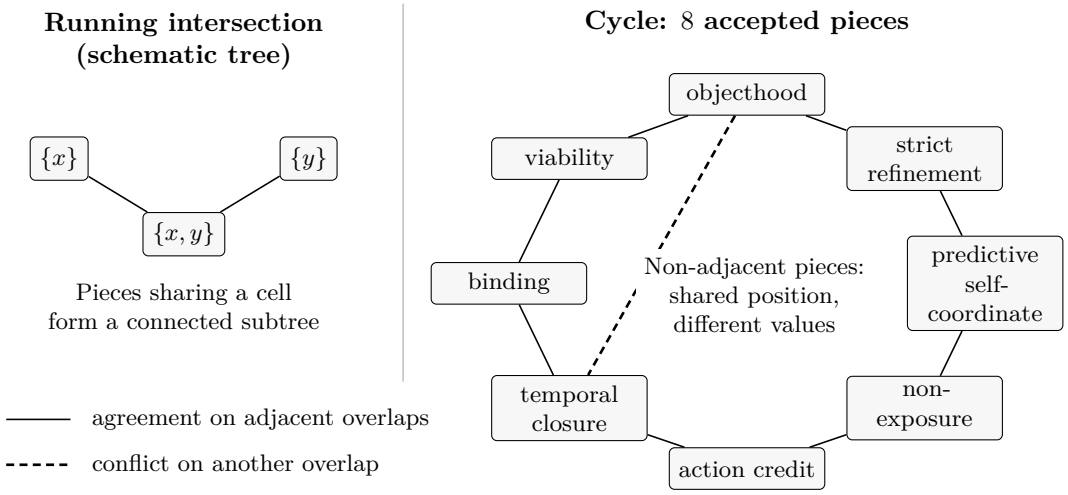


Figure 2. Gluing and its failure. On the left, when the pieces that share a cell form a connected subtree, agreement on adjacent overlaps glues. On the right, eight accepted pieces, one per condition, agree on every adjacent overlap, but two non-adjacent pieces read the same position table with different values, so no global assignment exists. The cycle is incidental, and the failure is one of running intersection.

We prove it by propagating equality along the paths that running intersection provides. The acceptance of the global assignment is not a conclusion of the theorem. It comes from each piece's own acceptance, which the realization check records separately. The same idea is standard for acyclic database schemes, where pairwise consistency gives global consistency exactly when the scheme is acyclic (Beeri et al., 1983, Theorem 3.4).

On the models of Section 5, a realization on the target is built in stages. Typed maps carry each condition's cells from the source to the target. Each mapped piece is then accepted, independently, by its own condition on the target, and a compatible cover on the target, also checked independently, glues the pieces. The assembly uses no premise about lineage or autonomy. Realization, in other words, can be established without knowing how the target came to be.

We cannot drop running intersection.

Proposition 1 (Agreement between neighbours is not enough)**.** *On the target of the two-copy model there are eight local assignments, one for each condition, each accepted and gated by its own condition, such that adjacent pieces agree on every cell they share, where adjacency forms a cycle through the eight conditions. Yet no global assignment restricts to all eight.*

The obstruction is local and simple. Two pieces that are not adjacent both read the table of positions and disagree on one entry, and neither neighbour of one of them reads that table. So the scope of that table is not connected, and running intersection fails. The cycle plays no role: a path of pieces with the same scopes would fail in the same way. What matters is that checking agreement only between neighbours misses a disagreement that running intersection would have exposed.

Public observations also lose information that the checks use. This is the formal version of the public shadow of Section 3.

Proposition 2 (Public shadow)**.** *On the target of the two-copy model there are two states that give identical readings on every channel of the fixed public interface, while their predictive readouts differ.*

The interface here is the fixed list of public channels of the two-copy model. The proposition says nothing about every possible observation.

4.2 Lineage

We check lineage one handoff at a time. A *handoff* is a finite sequence of stages from a source system to a distinct target system. Each stage records which systems are live in it and a frontier of the eight temporal fields of Section 2.

Definition 2 (Strict handoff)**.** A handoff is *strict* if it passes every check of the formal atomic gate, expanded in Appendix B.5. In summary, the gate requires the following.

- Source and target are distinct, and there are at least three stages.
- Only the source is live at entry and only the target at exit.
- Some stage has both live: this is the overlap.
- Every stage is realized, so there is no gap.
- Each step from one stage to the next increases order, tick, progress and records by a strictly positive amount, extends the route memory, and keeps an owner among the live systems.
- Events have unique identifiers, valid stages and owners, and their dependencies point forward.
- Every stage has a nonempty maintenance state.
- An intervention check shows that both systems' contributions are needed, and an audit check holds.
- The route is recorded as route-indexed.

A *strict chain* is a nonempty list of strict handoffs in which each handoff ends exactly where the next begins: the same intermediate system and the same boundary stage, including its whole frontier. Its first source and last target must differ.

Proposition 3 (Strict chains)**.** *Every strict chain admits an inductive derivation from its handoffs. Every stage of the concatenated chain is realized, and every step strictly increases order, tick, progress and records.*

The proposition repackages the checks. We include it because it shows that strict chains compose. This is composition of the chain check only. It is not composition of the full lineage check of Section 5, which also involves transport and coherence conditions. A chain through three distinct systems, with a handoff from the first to the second and one from the second to the third, is an instance (Section 5).

We also prove two facts about what a handoff carries. The eight fields of the temporal signature are transported exactly from source to target, on the checked frontier of histories, by a bridge that does not depend on which representative of a predictive class is chosen. Separately, the transported readouts are sufficient for every future observation, after words of every length. The record arrow is transported by any record map that preserves every strict step. A map that is merely monotone can fail: we give one that collapses a strict step and carries no arrow.

The examples of Section 3 now have exact statements.

Proposition 4 (Detached copy)**.** *There is a source state and a target state with equal predictive readouts, such that the target passes the realization and autonomy checks of the two-copy model, while for the detached sequence of stages, one with only the source live and one with only the target live, no strict handoff exists.*

Proposition 5 (Gap)**.** *On the two-copy model, if one overlap stage is removed from the realized episode, the source and target stages remain realized, the steps remain strict, the two routes' endpoints agree, and source and target have equal predictive readouts at the endpoint. Yet the episode fails the no-gap condition, and the handoff is not strict.*

Remark 1 (Re-encoding)**.** A nonidentity relabelling of the target's states exists. But a strict handoff requires its source and target to be distinct systems, so no relabelling of a single system is a handoff. This is a matter of typing, not a theorem about relabellings.

4.3 Autonomy after withdrawal

For autonomy we ask whether the target can keep itself going once the source, the bridge, any external controller, decoder or scheduler, and any undeclared repair have been removed. We check it with an invariant certificate. The certificate below is the simple finite form. Section 5 uses a stronger form, stated over full run histories (Lemma 2).

Definition 3 (Autonomy certificate)**.** Fix a finite controlled system with disturbances and actions, a region of its states, and a list of withdrawn and retained resources. An *autonomy certificate* states the following.

- The withdrawals are made and fixed before evaluation.
- The target's endpoint lies in the region.
- Every state in the region realizes the target's conditions.
- Maintenance uses only retained resources, all owned by the target.
- For every state in the region and every disturbance, there is an action owned by the target whose result stays in the region, does not stall, and remains within budget.

Theorem 2 (Autonomy for every finite horizon)**.** *Given an autonomy certificate, for every finite sequence of disturbances there is a run from the endpoint that answers it with the target's own actions, stays in the region at every step, and ends in a state that realizes the target's conditions.*

We prove it by induction on the sequence of disturbances, the standard argument for viability kernels (Aubin, 1991; Blanchini, 1999). We note two points of reading. The theorem gives *some* run that stays safe, chosen by the certificate. It does not say that every run does, which matters only when the system's response is nondeterministic. And the horizon is any finite length, not infinite. The result is relative to the model. It does not establish that any physical resource is available indefinitely. On the two-copy model, a certificate obtained by projecting the runtime to a finite part of its state instantiates the theorem for every genuine entry state. Section 9 treats unbounded maintenance.

We give three countermodels that mark what autonomy needs.

Proposition 6 (Essential withdrawn resource)**.** *On the two-copy model, removing the target's own recharge empties the target's greatest viable set, and autonomy fails.*

The resource withdrawn here is the target's own recharge. We have no countermodel in which the withdrawn support is an external scheduler.

Proposition 7 (Finite success is not unbounded maintenance)**.** *For every horizon h there is a finite system that succeeds through h steps and fails at step $h + 1$.*

So no finite run of success, however long, establishes unbounded maintenance on its own. In the terms of program verification, staying in a region forever is a safety property, not a liveness property, because any failure shows at a finite step (Alpern and Schneider, 1985). The proposition says that no single finite horizon of checking suffices for every system. A similar caution is familiar from control theory, where reachable regions after finitely many steps need not settle the infinite-horizon question (Bertsekas, 1972). There the quantifiers differ: one fixed system is studied in the limit, whereas here each horizon has its own system.

Proposition 8 (A clock that does not progress)**.** *There is a finite clock whose tick phase is stable while genuine progress is absent.*

Proposition 9 (Required conditions cannot be averaged)**.** *There are two defect profiles with the same total defect sum, one accepted and one rejected, because the rejected one fails a required condition.*

The last two results say that a clock's regularity is not its progress, and that a summed score cannot stand in for a list of required conditions. For this reason we state each check in this section as a conjunction of exact requirements.

5 The continuation theorem

We now put the three checks together. In words: a handoff whose certificate passes has a target that realizes the conditions, a strict live-overlap lineage from source to target, and a target that keeps itself going once everything used for the transfer is withdrawn.

5.1 The certificate

We call a *handoff model* the following data: a source and a target system, typed maps that carry each condition's cells from source to target, a cover on the target, the stages and routes of the episode, a record of the execution, and a list of resources marked as withdrawn or retained. All of these are given as raw data. The predicates that the checks use are defined from that data, and no derived object is supplied separately: not the target's assignment, not the predictive classes, not the success flags. The one thing that is supplied is evidence, namely a proof that an invariance obligation holds, used by the autonomy check. Given an assignment of values to the source's cells, we compute the target's assignment by mapping the source's local pieces across and gluing them on the target's cover.

Definition 4 (Continuation certificate)**.** A handoff model with a source assignment passes the *continuation certificate* if every check of the formal gate holds. Appendix B.5 expands the gate component by component. In summary, the gate requires the following.

1. *Source.* The source assignment satisfies the eight conditions on the source.
2. *Realization.* The mapped pieces satisfy the cover, scope, totality and value-preservation checks. Each is accepted by its own condition on the target, and adjacent pieces agree where they overlap.
3. *Lineage.* For every route of the episode the computed handoffs form a strict chain (Definition 2). The predictive and record readouts of source and target are related by the declared transport. Within each of the two systems, states with the same predictive code have the same observation laws for every finite sequence of actions. The translation from source to target commutes with initial states, transition weights, successors and observations on the whole set of source states

the execution reaches. The typed endpoint obligations and the coherence of the shared runtime also hold, including the temporal condition's decoder.

4. *Autonomy.* After every declared role is withdrawn, a game invariant certificate (Lemma 2) covers the target's runs. It is a stronger object than the finite certificate of Definition 3: it ranges over full run histories, and it requires normalized responses in the stated sparse format, legal moves using only the target's own retained resources, and strict advance of order, tick, progress and records. In the *realization-preserving* version the invariant includes the target's realization. In the *operational* version it requires only membership in a declared part of the target's state space.
5. *Bookkeeping.* The route is recorded as route-indexed, and the receiver of the source's content passes its own content and intervention checks.

5.2 The theorem

Theorem 3 (Continuation theorem)**.** *Let a handoff model with a source assignment pass the continuation certificate in its realization-preserving version. Then the following hold.*

(R) Realization. *The computed target assignment is total, restricts to every mapped piece on its scope, and is accepted by all eight conditions on the target.*

(L) Lineage. *For every route the computed handoffs form a strict chain. Moreover, within the source and within the target, any two states with the same predictive code give the same probability to every finite sequence of observations under every finite sequence of actions.*

(A) Autonomy. *After withdrawal there is a strategy for the target such that every history it generates, from every genuine entry state and following every outcome of positive probability, stays realized, uses only the target's own retained resources, makes only legal and affordable moves, and strictly advances order, tick, progress and records at every step.*

In the operational version, (R) and (L) hold as stated, and (A) holds with "stays realized" replaced by "stays in the declared part of the state space".

We prove the theorem from three lemmas, and we state the two that carry most of the weight.

Lemma 1 (From one step to every future)**.** *Consider a system with a finite list of outcomes, and suppose any two states with the same code satisfy three conditions. They have the same observation. For every action and every outcome, they move to states with the same code. For every action, they give equal total weight to each code class of next states. Then any two states with the same code give the same probability to every finite sequence of observations, under every finite sequence of actions.*

Lemma 2 (From an invariant to a strategy)**.** *Suppose an invariant set of runs satisfies the following conditions.*

- *The withdrawal holds, and some entry run exists.*
- *Every entry lies in the invariant.*
- *Every run in the invariant satisfies the realization predicate and the resource conditions.*
- *For every run in the invariant and every disturbance, some legal action has normalized outcome weights (nonnegative, summing to one, with a positive entry), and each outcome of positive weight leads to a run in the invariant and satisfies the transition law.*

Then there is a causal strategy under which every history generated from every entry consists of runs satisfying the realization predicate and the resource conditions. At each of these runs, for every disturbance, the chosen action is legal and normalized, and each of its positive outcomes satisfies the transition law.

Lemma 1 is a lumpability argument (Kemeny and Snell, 1976), carried from a single chain to a system with actions and observations, by induction on the length of the sequence. Its hypothesis makes the partition into codes a probabilistic bisimulation in the sense of Larsen and Skou (1991), adapted so that observations label the states. It is stronger still, since it aligns successor codes outcome by outcome and not only the masses of the code classes. Its conclusion is that states with the same code agree on every action-observation test in the sense of Littman et al. (2002). Lemma 2 is the standard construction of a controlled-invariant strategy (Blanchini, 1999; Aubin et al., 2011). In the terms of infinite games, the invariant is a trap for the environment, which chooses the disturbance and the outcome, and the strategy keeps play inside it (Mazala, 2002, p. 36). Its hypotheses exclude the vacuous cases: an empty set of entries, and empty responses. The third ingredient is the gluing theorem of Section 4.1, together with the locality of each condition, which carries local acceptance over to the glued assignment. With these in hand, the proof of the theorem consists of applying each lemma to its own check. We say this plainly, because the theorem is only as strong as its definitions. Its value lies in two facts. Each check's predicates are defined from the model's own data. Each conclusion is a property of the model, not merely a field of a certificate, even though autonomy draws on a supplied proof of invariance.

Two versions. In the realization-preserving version, autonomy implies realization, because the invariant includes it. In the operational version, the three conclusions are logically independent, in the following sense. On the two-copy model we make three single edits of the baseline: deleting one published cell, removing the source's support from one overlap step, or withdrawing the target's retained recharge. Each edit makes exactly one of the three conclusions fail while the other two hold. We do not exhibit all eight patterns of success and failure.

What the theorem does not give. We prove no converse: a target that realizes the conditions and continues the source need not have a passing certificate. The autonomy conclusion covers the disturbances of the recorded execution and every outcome of positive probability of the chosen response. We say nothing about resource shocks or perturbations of the transition weights. The sparse format in which responses are listed is a representation choice, and the certificate depends on it. Autonomy in our sense is equivalent to the existence of some invariant certificate. This is a statement about existence, not a procedure for finding one. The finite construction we use to produce certificates is sound but may be incomplete, so its failure to produce one would not show that autonomy fails. Lineage is strict lineage. It is not identity, and realization on the target is not the soundness of any certificate for consciousness. Transport of the temporal signature alone does not establish lineage.

5.3 Instances

Proposition 10 (A handoff that passes)**.** *On the two-copy model of Section 3, with its baseline assignment, the continuation certificate passes in both versions. Hence one handoff from the source copy to the target copy has realization on the target, strict lineage and autonomy after withdrawal.*

The handoff is a single change of system. Conditions (4) to (7) hold in the simplified form of Section 2. The autonomy check draws on the source's support in the recorded history, while the

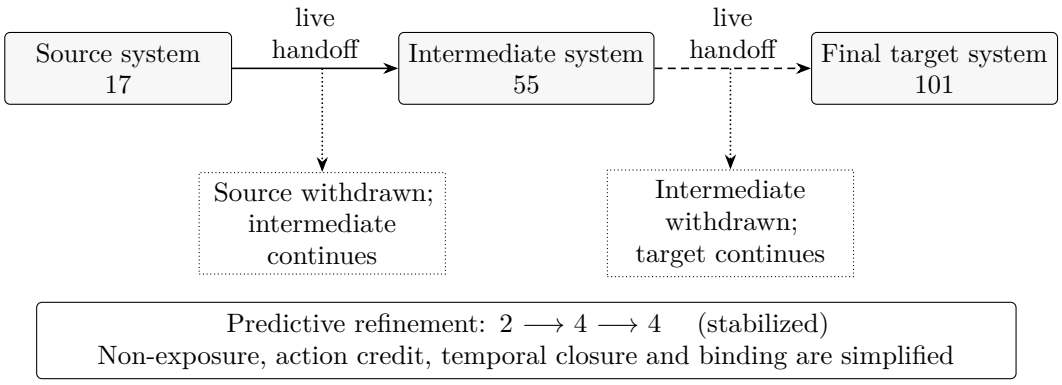


Figure 3. The chain through three systems. Each handoff is live, and each earlier system is withdrawn after its handoff. The box gives the predictive refinement on the model's finite factor. Conditions (4) to (7) hold in simplified form on this model.

lineage check uses the source's funded execution, and we keep the two apart. A second instance, with the histories of the two routes exchanged, also passes.

A single handoff is one step. The second instance is a chain.

Proposition 11 (A chain through three systems)**.** *On the chain model, an episode passes through three distinct systems, labelled* $17, 55, 101$ *in order, by two handoffs. For each of the* 8 *states of its finite factor, the following hold: the first and last systems are distinct; the final target owns itself; both handoffs realize the conditions on their targets; both satisfy their lineage checks; and the final target's controller wins its game after withdrawal. The middle system's whole signature agrees exactly between the two handoffs.*

The composite lineage here is the conjunction of the two handoffs' lineages. The general fact that strict handoffs compose into a strict chain is Proposition 3. On the chain model, conditions (4) to (7) again hold in simplified form, and disturbances are limited to a unit class. Several of the withdrawn roles coincide in this model. Its receiver preserves the payload and the source's classes, but not which history or route the source took. A general composition statement for such certificates also holds, but it is a formal rearrangement that does not use the agreement between the two handoffs, so we rely on the checked instance instead. Figure 3 draws the chain.

6 Which route? Memory, holonomy and the present

A transfer can reach the same target state by different routes. We ask what the target must remember about the route, and whether there is a present that does not depend on it. The answers are simple, and we give them because a criterion that ignores the route can fail in exactly these ways.

The current state need not be enough. Two histories can end in the same current state and still differ in what they predict. Then no rule that reads only the current state can predict both correctly. The observation is elementary.

Proposition 12 (Current state is not always sufficient)**.** *If two routes have the same current code and different futures, then no function of the current code returns both futures. On the two-copy model there are two such histories: the same current state, with different futures once funding is taken into account.*

What repairs the failure? If the pair of current code and route memory is enough to predict the future, then either the current code alone is already enough, or the route memory separates every pair of histories that share a current code and differ in their futures. So when two histories collide in this way, a target needs information beyond its current code, and route memory is one way to supply it. In a given domain, a licence of the kind below shows that it is not needed.

When route memory may be dropped. Such an argument can take one of three standard forms (Newman, 1942; Baader and Nipkow, 1998). In each, the declared rewrites of routes preserve the boundary and the executed actions; without that, a shared normal form says nothing about endpoints. In summary:

- *Normalization*: rewrites lower a rank, and every route reaches one shared normal form.
- *Confluence*: any two routes can be rewritten to a common one.
- *Flatness*: each event acts by an invertible forward clock. Critical pairs exist, cover all routes and are completed by valid completion rows. Routes are connected under completion moves, and rewrites reach a shared normal form. The completion rows satisfy their action equations, and the clock's holonomy around each completion row is the identity.

The complete hypotheses are those of the Lean sufficiency theorems indexed in Appendix C.

Theorem 4 (Licences for dropping route memory)**.** *Let a finite system of routes come with a comparison domain: routes with the same boundary, admissible and closed under the declared rewrites. If the domain carries a normalization, confluence or flatness licence, then any two routes in the domain reach the same endpoint.*

Each licence is sufficient, and we prove nothing more. In particular, a licence is not necessary. On a fixed domain of three one-event routes, two routes have equal endpoints and no licence holds. So equal endpoints alone do not show that the route system is flat. The flatness licence we use is the stronger adopted form: completion ranks and a shared unique normal form, not boundary equality alone.

Holonomy is not an arrow. Path dependence around a loop, holonomy in the broad sense (Nakahara, 2003), can look like a direction in time. It is not one.

Proposition 13 (Holonomy without an arrow)**.** *There is a finite route system with two reversible route maps that do not commute, and whose record update has no strict step. It has nontrivial holonomy and no endogenous arrow.*

No present fixed by the boundary. Finally, suppose routes share a boundary, and we ask for a "present" that depends only on that boundary. Holonomy can rule this out.

Proposition 14 (No canonical present)**.** *There is a two-phase clock with a translation that moves the designated present, phase zero, for which the local present along the identity route and along the loop cannot be written as a function of the shared boundary.*

The argument is one line: the two routes share the boundary, but their presents differ. We do not claim more, and in particular nothing about routes with genuinely different boundaries or about time as experienced. The lesson for continuation is modest but definite. A target that inherits its present from a transfer inherits it relative to a route. Keeping route memory is one way to handle

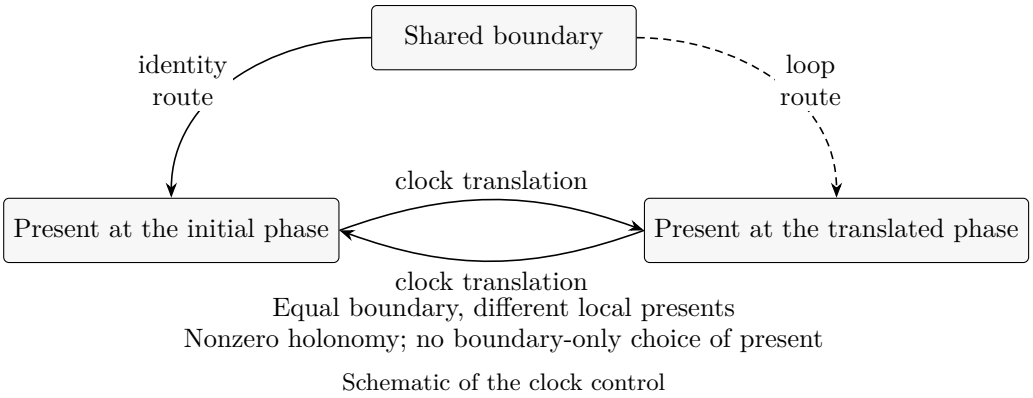


Figure 4. The clock control behind Proposition 14. Two routes share a boundary, but the loop translates the clock's phase, so the local presents differ and no present is fixed by the boundary alone.

this. A normalization, confluence or flatness licence is a sufficient reason to drop it, and it is not the only possible reason. A holonomy that moves the designated present rules out a present fixed by the boundary alone. A nontrivial transformation that leaves the designated present in place does not.

7 One source, two continuers

The fission cases of personal-identity theory ask what happens when one source divides into two equally good successors (Parfit, 1984; Olson, 2023). We cannot settle that question and do not try. We can say exactly what the structural checks of this paper do in such a case. The short answer is that continuation becomes a relation with two continuers, and nothing in the structure picks one of them.

A symmetric fork that passes. The chain model of Section 5 also has a fork. One source system hands off to two target systems, and both branches receive identical inputs, so the fork is symmetric by construction.

Theorem 5 (Continuation is relation-valued)**.** *In the symmetric fork of the chain model, labelled* $17, 201, 202$ *(source first), the following hold.*

1. *The source continues to each of the two targets, and the valid targets are exactly these two. Each carries the chain model's handoff certificate, covering realization, lineage and autonomy after withdrawal, for every state of the finite factor.*
2. *The swap of the two targets preserves validity and is an involution.*
3. *No continuer is fixed by the swap.*

The substance of the theorem is the first part. Each branch carries the chain model's handoff certificate: its realization check (both of its closed condition checks accepted, faithful receiver content, an actual entry at the target's root), its lineage check (future-transport laws, a passing handoff, raw transport), and a controller that wins its game after withdrawal. This is a kernel-checked computation on the model. It is the model's own certificate, not the full generic gate of Definition 4. The third part follows at once, since the swap exchanges the only two continuers. Its consequence is the one that matters. No rule that commutes with the swap can select a single continuer, because the swap fixes neither. This does not make either choice invalid: choosing one target alone still gives a valid continuation. What fails is any ground in the structure for the choice.

A smaller fork shows the same thing with less machinery.

Proposition 15 (Symmetry gives no ground for choosing)**.** *On the two-copy model there is a symmetric fork into two distinct target systems, both passing the realization and autonomy checks, in which the swap maps valid continuers to valid continuers and fixes none of them.*

Again the argument against a symmetric choice is elementary: the swap has no fixed point at all. We include the proposition because it shows that symmetric forks already arise for models far simpler than the chain.

Ancestry does not decide identity. One might hope to recover a single continuer from ancestry, that is, from which target descends from which source. On the chain model, after the fork, we can test this directly, with ancestry computed from the actual execution as in Section 8.

Theorem 6 (Ancestry underdetermines identity)**.** *On the chain model, for every n, there is a safe execution of $n + 1$ steps after withdrawal, ending in a fission, in which the source has two descendants holding the same state. That state passes the model's realization check, with conditions (4) to (7) in simplified form, and obeys its continuation law. Consider identity relations that are equivalences, that identify the source with at most one other node, and that identify the source with no node other than itself that is not its descendant under the computed ancestry. Among these there are two that identify the source with different descendants, and neither identification is forced.*

The identity notion here is deliberately weak and explicit: an equivalence relation that pairs the source with itself and with at most one other node, which must be a descendant. The theorem says that ancestry, even computed from an actual execution, does not choose between the two descendants. It does not say that neither is the source, nor that identity is undefined. A stronger statement, that ancestry rules out identity under every possible notion, is false as stated, and we do not make it.

These results fit the main positions in the literature without deciding between them. On the closest-continuer theory a tie leaves no continuer at all (Nozick, 1981), which is consistent with our result that no symmetric rule picks one. Such a theory makes whether one candidate continues the source depend on whether a rival exists. That is the standard criticism of it, and Hawley (2005) examines what exactly is wrong with it. On the multiple-occupancy view two persons share their stages before the division (Lewis, 1976), and we have instead let the continuation relation have two values. Cerullo (2015) argues informally that uploading yields branching continuers with no reason to favour either. The theorems above are formal structural analogues of that view on finite systems. Our identity notion, however, is constrained by ancestry and is shown not to decide, whereas his criterion is built from ancestry and is taken to settle continuity. Weber (2025a) states the same structure for uploading: the continuity relations that underlie persistence can hold from one to many. He argues that it is plausible that we could become several distinct persons, and we take no view on that. Nothing here is a claim about persons.

Branches preserved, recombination lost. Fission raises a converse question: if each branch is preserved, is the way the branches combine preserved too? It need not be.

Proposition 16 (Branchwise preservation is not recombination)**.** *On the two-copy model, the staged map preserves every ordered branch record from source to target, yet two things fail. On the target, the two assemblages have equal joint couplings, so joint content no longer strictly refines the branch records. And the source's and the target's joint couplings differ, so the staged transport of the binding condition fails.*

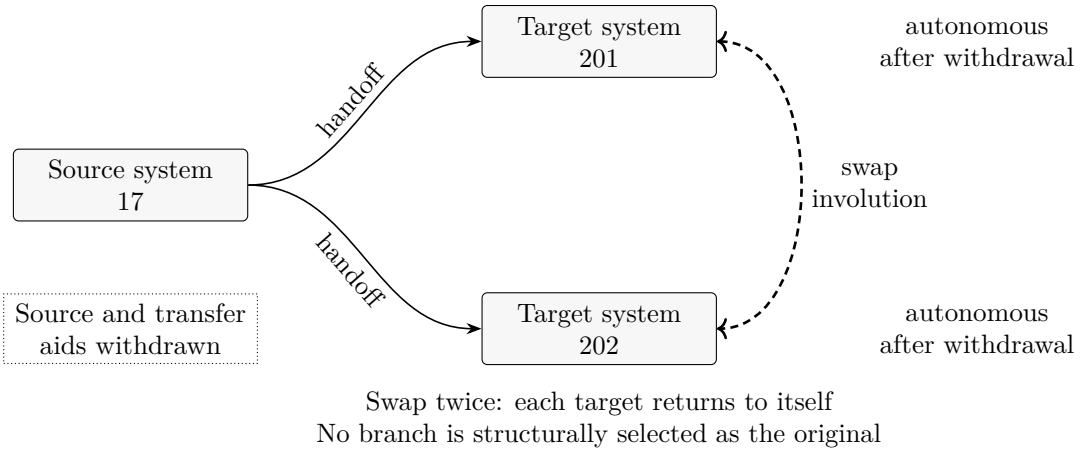


Figure 5. The symmetric fork. One source hands off to two targets that receive identical inputs, and each is autonomous after withdrawal. The swap of the two targets is an involution and fixes neither, so nothing in the structure selects one of them. We draw no conclusion about the identity of either branch.

The binding condition here is in its simplified form, a declared coupling, so the proposition concerns that form. It is the cross-system counterpart of a familiar point: aggregating branch contents does not determine their combination (Tsiokos, 2026j). Figure 5 draws the fork and the swap.

8 Continuation by ancestry, on a model that computes all eight conditions

Strict lineage asks for live overlap. A weaker requirement asks only that the target descend from the source through the recorded execution, and that the target's own predictive future law agree with its temporal frame. This is not equality with the source's future: a child's predictive class can already differ from the source's after one step. We call continuation under this weaker requirement *continuation by ancestry*. We prove it on a model where all eight conditions are computed together, and we treat fission and fusion there. The result is weaker than the continuation theorem in its lineage, and stronger in the grade of the conditions. No model in this paper has both.

8.1 Ancestry

We first collect what ancestry does and does not give. A *predictive bridge* between source and target descends to a map between their predictive classes. That map is unique and does not depend on the choice of representatives. Starting from admissible seeds of provenance, the records of an execution generate a least ancestry relation. Being least, it gives no ancestry from nothing. Two consequences follow.

Proposition 17 (Independent twin)**.** *In the declared operational controls, an independently constructed twin that is structurally equivalent to the source has no ancestry from it, however the execution is later extended by the controls' native and fusion steps.*

Proposition 18 (Reconstruction versus twin)**.** *In the declared operational controls, suppose a reconstruction is derived from the source after the source becomes inactive, passing the controls' snapshot, erasure, gap, activation and bridge-law checks, and an independent twin has the same content. Then the source is an ancestor of the reconstruction and not of the twin.*

Here the source becomes inactive, but its records are retained. The proposition does not describe total erasure of the source's information. A stronger formulation, in which ancestry runs from one target to another, is false, and we use the rooted form above.

Ancestry is not realization.

Proposition 19 (Ancestry does not give realization)**.** *On the chain model there is an edit that removes the target's own credit edges. It leaves the input, the bridge and the computed ancestry unchanged, while the target's realization check fails. Before the edit, the same check passes.*

So whatever lineage criterion one adopts, realization must be checked on its own.

8.2 The fusion model

The *fusion model* has a finite physical state space of 32 states, with two installed programs, addresses, tokens and accounts. Its execution histories grow without bound. On it, all eight conditions are computed jointly from a single installed execution, at interfaces fixed before evaluation. No condition is given an independent witness. Three choices are stated.

- Non-exposure, condition (4), is for the current family of observations on the maintained part of the state space. It is not privacy against every future observation.
- Action credit and temporal closure, conditions (5) and (6), use a process criterion. A target qualifies once its current residence has completed a local directional cycle, and its whole current epoch stays authenticated. The links to the current generator, ownership and control are mandatory.
- Objecthood, condition (1), is checked on the finite physical matrix, with the growing histories excluded from stationarity.

For autonomy, a certificate on a finite part of the state space, of 64 states, is lifted to every full history. The lift must be nonempty, cover every outcome of positive probability, and realize every response of the finite part.

We adopted these readings explicitly, in place of two stronger requirements. The first is that the record of completed exchanges end exactly at a cycle boundary. On the earlier shared-execution model that preceded this one, every immediate successor of a target fails that admission rule. The second is that a finite state space enumerate all growing histories. In general, no finite state space can carry an infinite strictly progressing run. Readers who reject the readings should read the theorem below as conditional on them.

Fusion. The model also merges branches. A *guarded fusion* combines two parents into a fresh output. It retires both parents, consumes a fresh capability, installs the common program and transfers all exclusive stock.

Proposition 20 (Fusion under ancestry)**.** *Under the least ancestry relation, fission, guarded fusion and empty outcomes are characterized as relations. On the fusion model, a guarded fusion has* 8 *settlement outcomes, with probabilities* $\frac{1}{40}, \frac{1}{40}, \frac{9}{40}, \frac{9}{40}, \frac{9}{40}, \frac{9}{40}, \frac{1}{40}, \frac{1}{40}$*, summing to* 1*. Each outcome retires both parents and yields a fresh output that has its own acquisition and a further continuation. When the parents' actions are opposed, the outcome is empty with probability* 1*.*

8.3 The ancestry theorem

Theorem 7 (Continuation by ancestry)**.** *On the fusion model, every request of the declared protocol that executes successfully yields, for the request's owner and from that same execution, the following:*

1. predictive ancestry from the source;

2. realization of all eight computed conditions;

3. autonomy after withdrawal that preserves that realization.

The successful requests include both children of a fission, after the source is withdrawn and each child has completed its own local acquisition, and the output of a guarded fusion after its acquisition. The same three properties hold for every history retained from an acquired fusion output.

The theorem is stated for successful requests, and we show separately that the children's requests and an output's request do succeed. A companion result covers histories retained after acquisition, in which the owner's steps are interleaved with other owners' steps. These histories are not themselves requests of the protocol. Its content lies in four places:

- invariance of acceptance, ownership and withdrawal along retained histories;
- the derivation of predictive ancestry from the execution;
- the derivation of the whole set of conditions from the installed program;
- the autonomy argument, in which the finite certificate and its lift simulate each other.

Several limits must be kept in view. Disturbances are a unit class, with renewable ports retained. The objecthood checks keep a recorded counterexample to a stronger, two-step uniqueness requirement. The model's full future laws, which include the initial observation, are distinct from the emissions of the branches after actions, and neither stands in for the other. Its greatest operational viable set includes charged residents that have not been admitted, while the greatest autonomy set that preserves the conditions compares only admitted competitors. The lineage here is predictive ancestry through recorded execution, which is weaker than live overlap. A freshly fused output does not yet realize the conditions. It is charged but not admitted, and it qualifies only after its own acquisition. Finally, the certificates for the source, the children and the fused output are certificates at different checkpoints of the execution, and we do not claim that they coexist.

What can be said about the architecture is this. We exhibit one designed model, with a finite physical state space and unbounded execution histories, on which the eight conditions, as adopted at interfaces fixed in advance, are computed jointly from a single execution, with the stated choices for conditions (1), (4), (5) and (6). This is not a certificate for the full architecture, whose complete panel of checks we do not verify. The architecture's integrated instance, which also requires a proof that the certificate is sound for what it is meant to capture, remains open (Tsiokos, 2026j). Figure 6 draws the model's timeline.

9 Other modes, and how far the results reach

9.1 Direct succession

Live overlap is one way for an episode to pass from source to target. Another is immediate. The last event of the source directly generates the first event of the target, with no stage at which both are live. We keep this mode separate.

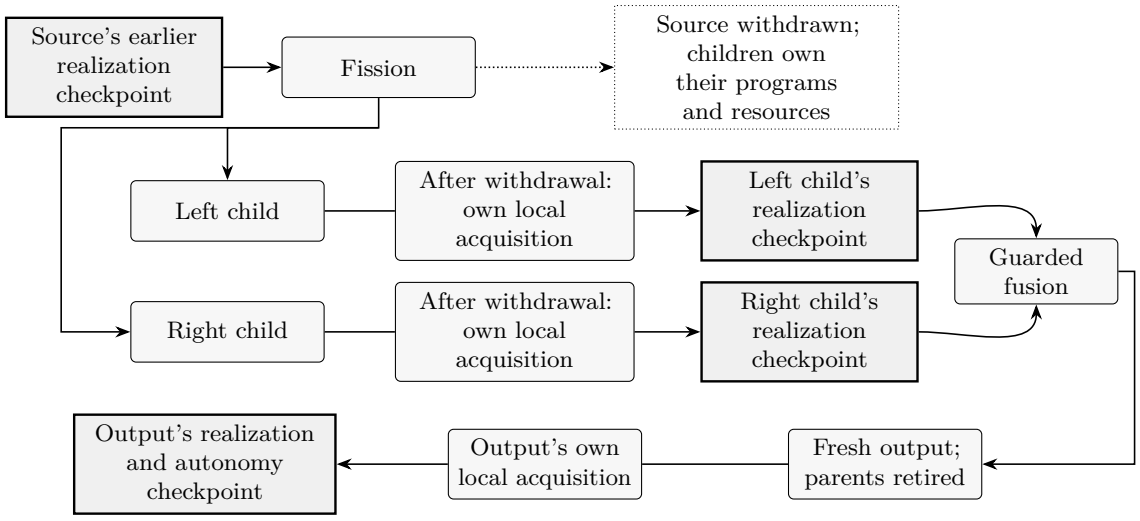


Figure 6. The timeline of the fusion model shows fission, the source's withdrawal, each child's own acquisition, guarded fusion with both parents retired, and the fused output's own acquisition. Heavy borders mark realization checks made at different points of the execution, and we do not claim that they hold together. The inset gives the fusion's settlement probabilities and the empty control.

Definition 5 (Direct succession)**.** A target event is a *direct successor* of a source event when the pair passes the computed direct-succession check (Appendix C). A direct successor is generated by that single, necessary source event and carries the source's whole eight-field frontier, advanced by one strict step. In addition, the source event is used once only, nothing is stored or replayed in between, there is no alternative parent, and route and ownership are transferred correctly.

Proposition 21 (Validity of direct succession)**.** *Every model in the computed direct mode yields a valid direct-successor certificate.*

The conditions listed in Definition 5 are necessary for passing the check. We do not prove that they are sufficient, so they summarize the check and do not characterize it.

Proposition 22 (What direct succession excludes)**.** *The direct criterion excludes four mechanisms: detached copies, replay from an archive, reconstruction through storage, and gaps. Reachability by data alone does not make a direct successor. Conversely, a separate data path can coexist with genuine direct succession.*

So the literal claim that direct succession excludes every data path is false, and we do not make it.

Proposition 23 (Two different modes)**.** *Strict overlap continuation and direct succession are incomparable. Each has instances that are not instances of the other. In fact the two modes are disjoint on every model, since overlap needs the source and target events in one stage and direct succession needs them in adjacent stages. Both embed in a broader, separately named relation of continuity paths, in which compatible mixed paths compose and closed paths are excluded.*

The broader relation never replaces strict continuation. When we say continuation without qualification we mean the strict, live-overlap kind.

9.2 Unbounded maintenance

Theorem 2 covers every finite horizon. To go further we need a policy that never runs out.

Theorem 8 (Unbounded maintenance on a finite factor)**.** *On the chain model, take the target's computed policy on a finite factor of its state, with unbounded event registers. Assume that the policy's inputs are ready, that the initial factor lies in the policy's computed greatest viable set, and that the initial balance covers the computed reserve. The policy's responses are owned by the target and normalized, and its positive outcomes advance and credit at least what they debit. Then for every admissible stream of disturbances and every stream of outcomes of positive probability, the resulting infinite run has the following properties at every step:*

- *it stays in the policy's greatest viable set and realizes the conditions, with (4) to (7) in simplified form;*
- *it strictly increases progress and records;*
- *it keeps the balance at or above the computed reserve.*

Unbounded registers are essential here. No infinite run that strictly progresses can fit into a finite full state, so the factor must be finite and the counters must not. Solvency rests on the stated balance of credit and debit. The greatest set is computed for the one fixed policy, not for every solvent strategy. Disturbances are a unit class. This is a model-relative result, not a theorem of physical persistence.

9.3 Exact defects and robustness

Required conditions cannot be averaged (Proposition 9). The positive counterpart is an exact semantics of defects.

Proposition 24 (Exact defect semantics)**.** *Let each condition carry a labelled rational defect, and require defects and tolerances to be nonnegative. A profile is accepted exactly when every required defect is zero and every soft defect is at most its own tolerance. Acceptance is downward closed, which means that lowering defects while keeping labels, types and tolerances preserves it. Acceptance is not determined by the total defect sum, since two profiles with the same schema and the same sum can differ.*

Proposition 25 (A bounded robustness box)**.** *In a rational-register extension of the two-copy model, on a fixed first event, every correction inside a closed five-component box of half-width $\frac{1}{2}$ preserves strictly positive advances and both complete handoff checks.*

This is robustness of a stated, narrow kind, with one extension, one event and one box. It does not return to the original integer model, and it does not cover arbitrary perturbations. A version stated for every perturbation of the original model is false.

10 Objections and replies

We state the strongest objections we know in the objector's own terms, and answer each.

"This is Parfit's fission case in new notation." The cases are Parfit's (Parfit, 1984), and we use them gladly. Here we pose the structural questions they raise as exact predicates on finite systems and check them in Lean. Parfit's Branch-Line case contrasts a replica's qualitative similarity with numerical identity. We set beside it a different contrast, between endpoint agreement and strict lineage. We do not identify the two contrasts, and nothing here bears on whether identity is what matters, on reductionism, or on persons at all.

"A perfect copy is a continuation." Under some criteria it is. Under the lineage check of this paper it is not, because no stage has source and copy live together (Proposition 4). We do not claim our criterion is the right criterion of personal identity, and we do not decide identity by stipulation. The claim is narrower and exact: endpoint agreement, realization and autonomy can all hold without the overlap that our lineage check requires. Anyone who counts the copy as a continuation must then say which check they are dropping.

"Your systems are tiny, so nothing here scales." The physical state spaces are finite and small, and the histories are unbounded. Every general result is proved for arbitrary types or arbitrary finite systems, and only the instances are small. The countermodels need to be small: one finite counterexample refutes a general shortcut. We do not generalize the exact results to noisy, large or continuous systems, and doing so would need its own approximate theory.

"You have shown that consciousness can be uploaded." We have not. Our results concern the structure described by the closure architecture, moved between finite systems. Whether satisfying that structure is being conscious is a separate question. The architecture paper answers it only conditionally, given its realization postulate (Tsiokos, 2026j). Substrate Indifference remains a postulate, and our results do not address it: moving structure between finite systems of different types is not an invariance theorem across substrates. The hypotheses linking structure to felt time and felt continuity are hypotheses, not conclusions of any result.

"The readings you adopted make the ancestry result easy." The ancestry theorem does rest on two adopted readings, and we state them where it is used (Section 8). Each replaces a stronger requirement that we do not use, for reasons stated where it applies. The first replaces a requirement on how records of exchanges must end. On the earlier shared-execution model, no immediate successor of a target is admitted under it. It asks instead for a completed local cycle with continuously authenticated residence, plus mandatory links to the current generator, ownership and control. The second replaces the demand that a finite state space list every growing history. It uses a finite certificate instead, lifted to every full history. A reader who rejects either reading should read Theorem 7 as conditional on it. The strict continuation theorem uses neither reading.

"A certificate that passes is not a certificate that is sound." Agreed. Passing our checks shows that the stated structural properties hold. Whether those properties are sound evidence for anything about consciousness is an open bridge, and we leave it open.

"Archives survive withdrawal, so the source is not really gone." Correct, and we say so. After withdrawal, the generated records of the source's history and of the births of its successors can remain. Our results infer neither erasure of the source's memory nor numerical identity from withdrawal.

"The record arrow is just a counter." In a sense, yes. The arrow in the temporal signature is an ordering by the growth of the system's own records, transported and checked as such. It is not a thermodynamic arrow and not an arrow defined by path reversal, and none of our results concerns those.

"Then nothing has been shown about the architecture as a whole." Something has, and it is stated exactly. One designed model computes all eight conditions jointly from one execution, at interfaces fixed in advance and with stated choices (Section 8). That model supports continuation by ancestry, not strict live-overlap continuation. The models that support strict continuation meet four conditions only in simplified form. No model here has both strict lineage and all eight conditions at design grade, and the architecture's integrated instance remains open.

11 Related work

Personal identity and its thought experiments. The question whether a copy, a teleported replica or a gradually replaced system is the same subject has a long literature (Williams, 1970; Parfit, 1971; Perry, 1972; Lewis, 1976; Nozick, 1981; Parfit, 1984). Olson (2023) surveys the fission problem and the two standard replies. The reduplication argument, that a relation which could hold between one earlier and two later persons cannot by itself make them identical, goes back to Williams (1957). On multiple occupancy, two persons share their stages before the division (Lewis, 1976). On non-branching criteria, a person survives only if no equally good competitor exists. Parfit discusses criteria of this kind, and Nozick's closest-continuer theory is a distinct proposal under which a tie leaves no continuer (Nozick, 1981). We do not take a position among these views. Our structural results sit beside them: under fission the continuation relation has two values, and no rule respecting the symmetry selects one (Section 7). Perry's notion of a branch, a maximal set of mutually related stages (Perry, 1972), invites comparison with our strict lineage. The two differ: our chains are checked sequences of handoffs with no maximality requirement. Gustafsson (2019) asks what form a non-branching clause should take when persistence rests on a continuity relation that can hold from one to many, and treats fusion as well as fission. Our relation holds between finite systems, is checked by a proof assistant, and is not analysed as identity.

Uploading and gradual replacement. Chalmers (2010) separates the question of organization from the question of identity, and distinguishes gradual, destructive and reconstructive uploading. Chalmers (1995) uses gradual replacement to argue for organizational invariance of experience. Cerullo (2015) argues informally that uploading can yield two continuers with no reason to favour either, under a criterion of branching identity. Bauer (2017) argues against branching identity. Corabi and Schneider (2012) ask whether an upload is at least a continuation of the original, in terms of its spatiotemporal and causal connection to it. In reply, Chalmers (2012) notes, with the example of a university that moved, that a moved thing may for a time exist at both places or at neither. He also notes that a gradually uploaded system stays connected to the original throughout. Weber (2025b) argues that gradual replacement and scan-and-copy differ metaphysically, in the immanent causation they involve. Our three questions formalize questions of this kind on finite systems, and we too keep identity out of them. We do not claim that gradual versus instant uploading corresponds to our distinction between overlap and direct succession. Our classes require operational checks that timing alone does not supply. And nothing here bears on whether an upload is conscious.

Temporal experience. Overlap models of the stream of consciousness, in which successive specious presents (James, 1890, ch. XV) share content, are the phenomenological counterpart of our live-overlap handoffs (Dainton, 2000, 2023). Dainton and Bayne (2005) propose that phenomenal continuity, rather than psychological continuity, is the guide to personal persistence, and Dainton (2008) develops that view and treats fission and fusion within it. Our overlaps are shared system state, not co-consciousness, and we adopt no model of experienced duration. We also take no side between views on which experience inherits the temporal structure of its objects (Phillips, 2014) and views on which it need not (Lee, 2014). Within integrated information theory, Comolatti et al. (2025) account for experienced time by moments that overlap in a directed way, and take the present to be a cause–effect structure specified by a system in its current state. Our predictive present is a different object, a field of the temporal signature carried across a transfer. We establish no bridge between the two and draw no conclusion about their account.

Mathematical consciousness science. Our setting is that of mathematical models of consciousness, where a model's objects and their well-definedness conditions are stated explicitly (Kleiner, 2020). In that framework experience is defined at an instant, and what counts as an instant is left to the model (Kleiner, 2020, Definition 2). Our checks concern a different object: how a temporal signature, read off each state, is carried and maintained over handoff episodes and future sequences of actions. Integrated information theory requires a unique maximal substrate. When the maximal candidates tie and all of them overlap, its stated rule picks the next-best unique system (Albantakis et al., 2023). That the theory's maximizations need not have unique results was already noted in its formalization (Kleiner and Tull, 2021, Section 12.2). This is a case from the field's own literature of a theory facing a uniqueness requirement over tied candidates. It is a different move from selecting among the tied ones, and our results do not show that the rule needs to break any symmetry. Substrate questions also appear as formal no-go arguments (Kleiner and Ludwig, 2024). Those concern which systems are conscious, whereas ours concern which systems count as structural continuers, so the two neither support nor contradict each other.

Mathematical precursors. The ingredients are standard, and we credit them where they are used:

- gluing over covers with running intersection (acyclic database schemes, Beeri et al., 1983, Theorem 3.4; see also Vorob'ev, 1962 and Abramsky, 2013). Global sections and their obstruction are treated in Abramsky and Brandenburger (2011), whose obstructions concern distributions. Our cyclic obstruction is instead a failure of running intersection.
- confluence and normal forms (Newman, 1942; Baader and Nipkow, 1998).
- bisimulation as behavioural equivalence (Park, 1981; Milner, 1989), and its probabilistic form (Larsen and Skou, 1991).
- lumpability of Markov chains (Kemeny and Snell, 1976).
- controlled invariance and viability (Blanchini, 1999; Aubin, 1991), and traps in infinite games (Mazala, 2002).
- safety and liveness properties (Alpern and Schneider, 1985).
- limits of finite-step reachability (Bertsekas, 1972).
- holonomy (Nakahara, 2003).

- predictive equivalence (Shalizi and Crutchfield, 2001) and predictive representations of state (Littman et al., 2002).

We claim no priority for any of these ideas, separately or together.

The Six Birds framework. The questions and the design of the models come from a broader programme, the Six Birds framework, which we describe in Appendix A together with the sources the design drew on. None of our results depends on it.

12 Conclusion

We asked what would make the result of moving a closure architecture to a different finite system a continuation rather than a copy. Our answer has three parts, each checked separately: realization on the target, strict live-overlap lineage, and autonomy after withdrawal. A certificate that checks them is sound, finite models pass it, and finite countermodels show what fails when a check is dropped or replaced by endpoint agreement. Continuation is a relation, and under symmetric fission nothing in the structure selects one continuer. A target inherits its present relative to the route the transfer took. On one designed model, all eight conditions, as adopted at interfaces fixed in advance, are computed jointly from one execution. A weaker continuation, by ancestry, holds there for both branches of a fission and for a guarded fusion, each after its own acquisition.

Limitations. We state them once.

- Every result is about finite physical state spaces, some with unbounded histories. We do not generalize to noisy, large or continuous systems.
- The strict continuation instances meet conditions (4) to (7) only in simplified form. The model that computes all eight conditions supports only continuation by ancestry, under two adopted readings. Its certificates sit at different checkpoints.
- Autonomy is model-relative. Disturbances are limited to declared classes, with no robustness to resource shocks, and nothing here establishes indefinite physical maintenance.
- Robustness is shown for one narrow box in one extension of one model.
- Nothing here shows that any measurement of a real system identifies these structures.
- Nothing here establishes consciousness, felt continuity, survival, numerical identity or Substrate Indifference.

Open problems.

- Exhibit a model with strict live-overlap lineage and all eight conditions at design grade, or prove that some stated obstruction prevents one.
- Prove that a certificate of the architecture is sound for what it is meant to capture, or give a reason why no such proof is possible.
- Extend autonomy beyond declared disturbance classes, to robustness against resource shocks.
- Find a converse for the continuation theorem: conditions under which a continuation that realizes the conditions must have a passing certificate.

Data and code availability

No empirical data were collected, fitted or used in this work. The Lean development, the finite models, the result snapshot and the scripts that check them are in a public repository, https://github.com/ioannist/six-birds-transcendence. The preprint's archival DOI is doi:10.5281/zenodo.23233188.

The development builds with Lean 4.28.0 (`leanprover/lean4:v4.28.0`) using its core library only. It has 562 modules and records 4586 distinct declarations in its inventory. It proves 25 of the 39 results it states and checks the other 14 as finite countermodels, and 0 are left open. The 82 declarations we cite depend only on the standard axioms `Classical.choice, Quot.sound, propext`. Our development imports no part of the vendored Six Birds library. That library contains a non-standard axiom on which none of our declarations depends. The finite models are specified in 15 model files.

In the verification recorded on 2026-10-13 and 2026-10-14, a clean build ran 799 Lean build jobs and succeeded, and the validator found no placeholder and no untrusted axiom. All transcription checks between model files and Lean passed, as did the validator's 17 self-tests and 205 paper tests. The repository also keeps the test suite of an earlier Python scaffold that the Lean development superseded. There, 130 tests pass and 6 fail, as they already did when the Lean development was completed. Five of the failing tests scan the superseded scaffold, and one tests that a historical file builder is idempotent. No result here depends on the scaffold.

Every computed value in our figures and tables is read from the snapshot and pinned to a Lean theorem that states it. The stage and callout numbers in the schematic Figure 1 are labels, not values. The kernel checks the Lean statements. That those statements say what our prose says is not a kernel fact. We checked it in a separate review, which set each formal statement in this paper beside the printed Lean statement of every declaration it cites, together with that declaration's axioms. The review used AI assistance, as disclosed below.

Disclosures. This preprint has not been peer reviewed. The repository's review records document an internal review loop driven by the author. The Lean formalization, the repository tooling, the preparation of the manuscript and the statement-by-statement review used AI assistance. The author is responsible for all claims.

A The Six Birds framework behind this paper

This appendix is the only place where we use the vocabulary of the Six Birds framework, a programme of structural laws for emergence and closure across layers. The paper's questions and the design of its models come from that programme, and this appendix says how. None of our results depends on any statement below. Each result stands on its own proof in Lean.

The question in the framework's terms. In the framework, a layer is promoted when a package of conditions closes on a carrier, and the consciousness project specifies such a package, the C-package with clauses C1 to C8 (Tsiokos, 2026b,j). The project behind this paper, there called substrate transcendence, asks when a promoted package can be carried from one carrier to another. It separates the three lanes the body calls realization, lineage and autonomy, and prices each separately. The strict criterion of the body is the framework's Temporal-FIACH certificate. Continuation by ancestry is its Temporal-IAPS fallback.

The sources the design drew on. The design of the models drew on statement-level sources in the framework's corpus:

- the C-package contract and its temporal clause (Tsiokos, 2026b);
- predictive equivalence, predictive transport and route residue on fixed support (Tsiokos, 2026e);
- branchwise versus recombination structure (Tsiokos, 2026f);
- the catalog laws for holonomy-memory repair, relation-valued object persistence and arrows from record monotonicity (Tsiokos, 2026d);
- holonomy without an endogenous arrow (Tsiokos, 2026h);
- flatness, local confluence and unique normal forms (Tsiokos, 2026c);
- time as order, measure and arrow, funded clocks, and clock translation (Tsiokos, 2026g);
- viability and target-owned maintenance (Tsiokos, 2026a);
- an analogy with logical objects in quantum error correction (Tsiokos, 2026i).

Each source carries its own scope warning. Most results are on fixed support and do not by themselves give a cross-carrier theorem. The catalog laws are conditional on declared quotients and instruments. The quantum analogy stays an analogy.

Versions. The design read these sources in the versions frozen on 2026-09-28, which we cite by content hash in the artifact. Several were revised afterwards. Ten of the nineteen cited statements have since changed, and three of the changes are material. The time paper now defines the arrow by path reversal and calls a stationary monotone ledger trivial. The agency paper no longer credits zero defect to repair. The holonomy paper no longer states that current transport exists exactly when the interface is flat. These revisions do not affect our results, which do not depend on the sources. They do explain one choice of wording in the body: the record arrow is an ordering by record growth, not an arrow in the revised time paper's sense.

What the framework does not supply. The corpus is not evidence that any system is conscious or that consciousness can move between substrates. Substrate Indifference, A2 in the architecture, remains a postulate with no transport or invariance theorem. Certificate soundness is open, and the architecture's integrated instance has not been exhibited.

B The finite models

We describe each model in words. Appendix C names the Lean definitions, and the artifact contains the full data. Every computed value below, such as a state count, a mass, a distance or a refinement trajectory, is read from the result snapshot and pinned to a Lean theorem that states it. The other small numbers, such as the numbers of stages, copies, bits and programs, describe how a model is specified, and the Lean definitions fix them.

B.1 The two-copy deadline model

The base is the deadline system of Tsiokos (2026k): a walker on a wheel of positions with a hidden two-valued phase, a deadline, and a viability kernel on which a predictive self-coordinate is forced. We take two copies, tagged source and target. With the lethal states, the tagged state space has 24 states, of which 4 are lethal. The handoff adds a position along the episode, a token budget with a recharge owned by each tag, and a register recording the route.

We run the episode through four stages: source live, two overlap stages in which both copies are live, and target live. Two checked routes lead from distinct source histories to the target.

The conditions hold as follows.

- *Objecthood*, condition (1), is the deadline system's staged closure kernel. At perturbation size $\frac{1}{16}$, the perturbed two-step kernel has a unique fixed point that keeps $\frac{77}{80}$ of its mass on the specified four-state core. Its total-variation distance from the uniform distribution on the core is $\frac{3}{80}$.
- *Strict refinement* and the *predictive self-coordinate*, conditions (2) and (3), are the deadline system's own, computed on each copy. The refinement trajectory is $6, 7, 7$.
- *Viability*, condition (8), is the greatest funded kernel. It is nonempty with recharge and empty without.
- Conditions (4) to (7) are simplified. Non-exposure is replaced by a single collision between two states that agree on the current channels and differ in their predictive state. Action credit is replaced by a declared three-cycle credit rule. Temporal closure is replaced by a declared order on events, with a record and a tick added for the purpose. Binding is replaced by a declared coupling between two assemblages.

The staged map carries all 8 conditions from source to target. A rational-register extension of this model carries the robustness result of Proposition 25.

B.2 The chain model

The chain model has a finite factor of 8 states, three bits for phase, visibility and a hidden bit, together with unbounded counters, ticks and histories. Its refinement trajectory is $2, 4, 4$.

- *The chain.* An episode passes through the systems $17, 55, 101$ by two handoffs.
- *The fork.* A symmetric fork hands off from the first system to two targets, $17, 201, 202$ with the source listed first, and both branches receive identical inputs.

Conditions (1), (2), (3) and (8) are computed from the model's own dynamics: a nonuniform fixed point, a computed split, a recursive refinement, and a greatest viable set that empties when recharge is cut. Conditions (4) to (7) are simplified, as above. Disturbances form a unit class.

B.3 The fusion model

The fusion model has 32 physical states and two installed programs, together with addresses, tokens of one unit each, and accounts. Its viability factor, the physical state together with the charge, has 64 states. The operational core kept under renewable supply has 16 states. The refinement trajectory is $9, 16, 16$.

Its execution histories grow without bound, and we never enumerate them. All eight conditions are computed from one installed execution, with the choices stated in Section 8. A guarded fusion has 8 settlement outcomes. We specified the model, and fixed its readings, before evaluating it.

B.4 Small controls

For several countermodels we use small dedicated systems:

- a source-only and a target-only stage, for the detached copy
- two reversible three-state route maps that do not commute, for holonomy without an arrow;
- a two-phase clock loop, for the absence of a canonical present
- a singleton clock, for a tick without progress
- pairs of defect vectors, for scalar summaries
- a relabelled two-branch fork, for the symmetric fork
- a countdown of length $h + 2$ for each horizon h, for finite success
- a domain of three one-event routes, for the licences
- controls separating direct succession from overlap and from data reachability.

B.5 The checks, expanded

The definitions in Sections 4 and 5 summarize the formal gates, and here we expand them component by component. The named Lean predicates, listed in Appendix C, are authoritative. This list explains them and does not replace them.

The strict handoff gate. We say that a handoff passes when all of the following hold.

1. Source and target are distinct, and there are at least three stages.
2. The entry stage's support is exactly the source and the exit stage's support is exactly the target, with matching owners.
3. Every stage is realized, has nonempty support, has its owner in its support, and has a nonempty maintenance state.
4. Some stage contains both systems.
5. Event identifiers are unique, and events have valid stages and owners.
6. Every dependency points forward in stage.
7. There is one readout update per adjacent pair of stages. Each update changes the predictive field by its specified nonnegative increment, and increases order, tick, progress and records by strictly positive amounts. It appends a nonempty extension to the route memory, keeps a nonempty maintenance state, and keeps the owner in the support.
8. An intervention check holds: either additive nonredundancy of the projections, or failure of the episode once mixed support is removed. An audit check also holds.
9. The route status is route-indexed.

A chain of handoffs passes when each handoff passes, each pair of consecutive handoffs agrees exactly at the join (the same intermediate system and the same boundary stage, including its whole frontier, route memory and owner), and the first source differs from the last target.

The continuation gate. We say that a handoff model with a source assignment passes the gate in its realization-preserving version when all of the following hold.

1. *Raw laws.* The recipe for the viability condition equals the fixed rule, and the resource runtime's presence and requirement instructions agree with the nominal recipe.
2. *Protocol.* The withdrawal metadata decodes, and a finite check of the decoded rows, policy, ticks, account record, routes and writers passes.
3. *Readiness.* The acquisition and resource inputs are ready, and any missing consumed input is rejected, not defaulted.
4. *Domain.* The required domains are nonempty, and the disturbances, actions, outcomes, factors and resources are each enumerated completely by finite lists.
5. *Source.* The source assignment satisfies the eight conditions on the source.
6. *Coherence.* The owner map is injective. The system and schema checks pass. The temporal condition's rule is the temporal decoder. The presentation is coherent, and so is the target's execution.
7. *Realization.* For the mapped pieces, the cover, scope, totality, reference and value-preservation checks pass, and every source cell is covered. Every condition accepts its mapped piece on the target, and the mapped pieces agree on every edge of the cover.
8. *Lineage.* The complete lineage check passes, together with the typed endpoint check. For every route the computed legs form a strict chain. The seed, factor transport, record transport and retained route readouts pass. The execution frontiers of both systems satisfy the semantic readout, including sufficiency of the predictive code for every finite sequence of actions within each system. The translation commutes with initial states, transition weights, successors and observations on the whole reached set of source states. Both frames share the runtime schema, and the execution links pass.
9. *Autonomy.* A game invariant certificate exists in the sense of Lemma 2, with these instantiations:
   - the entries are the registers reached by one step after a genuine arrival;
   - the withdrawal removes all six declared roles, and keeps removed and retained resources disjoint;
   - the realization predicate is realization together with membership in the declared region and acceptance of the run's assignment;
   - the resource conditions require target ownership, retention and a positive balance of every required resource, with the source absent and a writer present;
   - legal moves are owned by the target, use only retained resources, and are affordable;
   - the transition law requires resource closure, exact balance updates, and strict advance of order, ticks, progress and records.

   Responses are listed in a sparse format. At every generated play and for every disturbance, every listed weight of the selected response is positive. Unselected actions are not constrained.
10. *Receiver and scope.* The receiver's content and intervention laws hold, the endpoint is active, the route status is route-indexed, and recognition is recorded as not addressed.

The operational version is identical, except that the realization predicate in the autonomy check is replaced by membership in the declared part of the state space.

The finite autonomy certificate of Section 4.3. We state this certificate for a finite controlled system. Its fields record the withdrawals, and record that the boundary and the disturbance class were fixed before evaluation. It also requires the following:

- the endpoint lies in the region;
- realization holds in the region;
- maintenance resources are retained and owned by the target;
- for every state in the region and every disturbance, some action owned by the target leads to a state in the region without stalling and within budget.

C Result-to-Lean index

Result / title	Registry	Headline Lean declarations	Axioms	In inv.
definition 1 Cover with running intersection	TTH-01	(a) `CSTTT.PackageTheory.acyclicLocalWitness Gluing`	`Classical.choice`	(a) yes
theorem 1 Gluing	TTH-01	(a) `CSTTT.PackageTheory.acyclicLocalWitness Gluing`	`Classical.choice`	(a) yes
proposition 1 Agreement between neighbours is not enough	TTN-08	(a) `CSTTT.POS04.Assigned.ActualCyclePrototype. actual_pos04_cyclic_obstruction`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 2 Public shadow	TTN-02	(a) `CSTTT.POS04.ttn02_public_shadow`	`Quot.sound, propext`	(a) yes
definition 2 Strict handoff	TTH-04	(a) `CSTTT.CorridorTheory.AtomicChain.strict TemporalCorridorComposition_generic`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 3 Strict chains	TTH-04	(a) `CSTTT.CorridorTheory.AtomicChain.strict TemporalCorridorComposition_generic`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 4 Detached copy	TTN-01	(a) `CSTTT.RealControls.ttn01_detached_copy` (b) `CSTTT.RealControls.ttn01_no_atomic_ corridor` (c) `CSTTT.RealControls.ttn01_exists`	`Classical.choice, Quot.sound, propext`	(a) no (b) yes (c) yes
proposition 5 Gap	TTN-09	(a) `CSTTT.POS04.ttn09_gap_resume`	`Quot.sound, propext`	(a) yes
remark 1 Re-encoding	TTN-13	(a) `CSTTT.POS04.ttn13_same_carrier_boundary`	`propext`	(a) yes
definition 3 Autonomy certificate	TTH-05	(a) `CSTTT.AutonomyTheory.postWithdrawalFinite HorizonInvariance` (b) `CSTTT.TargetTheory.GenericHandoff.POS04 HistoricalRuntimeInstance.positive_each_ entry_exact` (c) `CSTTT.TargetTheory.GenericHandoff.POS04 HistoricalRuntimeInstance.positive_each_ entry_words`	(a) `none` (b) `Classical.choice, Quot.sound, propext` (c) `Classical.choice, Quot.sound, propext`	(a) yes (b) no (c) no
theorem 2 Autonomy for every finite horizon	TTH-05	(a) `CSTTT.AutonomyTheory.postWithdrawalFinite HorizonInvariance` (b) `CSTTT.TargetTheory.GenericHandoff.POS04 HistoricalRuntimeInstance.positive_each_ entry_exact` (c) `CSTTT.TargetTheory.GenericHandoff.POS04 HistoricalRuntimeInstance.positive_each_ entry_words`	(a) `none` (b) `Classical.choice, Quot.sound, propext` (c) `Classical.choice, Quot.sound, propext`	(a) yes (b) no (c) no
proposition 6 Essential withdrawn resource	TTN-06	(a) `CSTTT.POS04.ttn06_withdrawn_essential_ resource`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 7 Finite success is not unbounded maintenance	TTN-12	(a) `CSTTT.RealControls.ttn12_exists_for_ every_horizon`	`Quot.sound, propext`	(a) yes
proposition 8 A clock that does not progress	TTN-07	(a) `CSTTT.RealControls.ttn07_exists`	`Quot.sound, propext`	(a) yes

Continued on next page

Result / title	Registry	Headline Lean declarations	Axioms	In inv.
proposition 9 Required conditions cannot be averaged	TTN-10	(a) `CSTTT.RealControls.ttn10_equal_scalar_exists`	`propext`	(a) yes
definition 4 Continuation certificate	TTH-06	(a) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.temporalFIACHCertificateSoundness_generic_F` (b) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.temporalFIACHCertificateSoundness_generic_O` (c) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.full_autonomy_implies_package` (d) `CSTTT.TargetTheory.GenericHandoff.POSO4FinalHandoff.temporalFIACH_three_lane_separation` (e) `CSTTT.TargetTheory.GenericHandoff.StrategyGame.game_invariant_completeness` (f) `CSTTT.TargetTheory.GenericHandoff.PackageEngine.constructs` (g) `CSTTT.TargetTheory.GenericHandoff.FutureLumping.all_words` (h) `CSTTT.TargetTheory.GenericHandoff.StrategyGame.invariant_game_soundness`	(a) `Classical.choice, Quot.sound, propext` (b) `Classical.choice, Quot.sound, propext` (c) `Classical.choice, Quot.sound, propext` (d) `Classical.choice, Quot.sound, propext` (e) `Classical.choice, Quot.sound, propext` (f) `Quot.sound, propext` (g) `Classical.choice, Quot.sound, propext` (h) `Classical.choice, Quot.sound, propext`	(a) yes (b) yes (c) yes (d) yes (e) no (f) no (g) no (h) no
theorem 3 Continuation theorem	TTH-06	(a) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.temporalFIACHCertificateSoundness_generic_F` (b) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.temporalFIACHCertificateSoundness_generic_O` (c) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.full_autonomy_implies_package` (d) `CSTTT.TargetTheory.GenericHandoff.POSO4FinalHandoff.temporalFIACH_three_lane_separation` (e) `CSTTT.TargetTheory.GenericHandoff.StrategyGame.game_invariant_completeness` (f) `CSTTT.TargetTheory.GenericHandoff.PackageEngine.constructs` (g) `CSTTT.TargetTheory.GenericHandoff.FutureLumping.all_words` (h) `CSTTT.TargetTheory.GenericHandoff.StrategyGame.invariant_game_soundness`	(a) `Classical.choice, Quot.sound, propext` (b) `Classical.choice, Quot.sound, propext` (c) `Classical.choice, Quot.sound, propext` (d) `Classical.choice, Quot.sound, propext` (e) `Classical.choice, Quot.sound, propext` (f) `Quot.sound, propext` (g) `Classical.choice, Quot.sound, propext` (h) `Classical.choice, Quot.sound, propext`	(a) yes (b) yes (c) yes (d) yes (e) no (f) no (g) no (h) no

Continued on next page

Result / title	Registry	Headline Lean declarations	Axioms	In inv.
lemma 1 From one step to every future	TTH-06	(a) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.temporalFIACHCertificateSoundness_generic_F` (b) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.temporalFIACHCertificateSoundness_generic_O` (c) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.full_autonomy_implies_package` (d) `CSTTT.TargetTheory.GenericHandoff.POS04FinalHandoff.temporalFIACH_three_lane_separation` (e) `CSTTT.TargetTheory.GenericHandoff.StrategyGame.game_invariant_completeness` (f) `CSTTT.TargetTheory.GenericHandoff.PackageEngine.constructs` (g) `CSTTT.TargetTheory.GenericHandoff.FutureLumping.all_words` (h) `CSTTT.TargetTheory.GenericHandoff.StrategyGame.invariant_game_soundness`	(a) `Classical.choice, Quot.sound, propext` (b) `Classical.choice, Quot.sound, propext` (c) `Classical.choice, Quot.sound, propext` (d) `Classical.choice, Quot.sound, propext` (e) `Classical.choice, Quot.sound, propext` (f) `Quot.sound, propext` (g) `Classical.choice, Quot.sound, propext` (h) `Classical.choice, Quot.sound, propext`	(a) yes (b) yes (c) yes (d) yes (e) no (f) no (g) no (h) no
lemma 2 From an invariant to a strategy	TTH-06	(a) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.temporalFIACHCertificateSoundness_generic_F` (b) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.temporalFIACHCertificateSoundness_generic_O` (c) `CSTTT.TargetTheory.GenericHandoff.FinalHandoff.full_autonomy_implies_package` (d) `CSTTT.TargetTheory.GenericHandoff.POS04FinalHandoff.temporalFIACH_three_lane_separation` (e) `CSTTT.TargetTheory.GenericHandoff.StrategyGame.game_invariant_completeness` (f) `CSTTT.TargetTheory.GenericHandoff.PackageEngine.constructs` (g) `CSTTT.TargetTheory.GenericHandoff.FutureLumping.all_words` (h) `CSTTT.TargetTheory.GenericHandoff.StrategyGame.invariant_game_soundness`	(a) `Classical.choice, Quot.sound, propext` (b) `Classical.choice, Quot.sound, propext` (c) `Classical.choice, Quot.sound, propext` (d) `Classical.choice, Quot.sound, propext` (e) `Classical.choice, Quot.sound, propext` (f) `Quot.sound, propext` (g) `Classical.choice, Quot.sound, propext` (h) `Classical.choice, Quot.sound, propext`	(a) yes (b) yes (c) yes (d) yes (e) no (f) no (g) no (h) no
proposition 10 A handoff that passes	TTH-06	(a) `CSTTT.TargetTheory.GenericHandoff.POS04FinalHandoff.positive_F` (b) `CSTTT.TargetTheory.GenericHandoff.POS04FinalHandoff.positive_O` (c) `CSTTT.TargetTheory.GenericHandoff.POS04FinalHandoff.positive_receiver` (d) `CSTTT.TargetTheory.GenericHandoff.POS04FinalReversal.full` (e) `CSTTT.TargetTheory.GenericHandoff.POS04FinalReversal.operational`	`Classical.choice, Quot.sound, propext`	(a) yes (b) yes (c) yes (d) yes (e) yes
proposition 11 A chain through three systems	TTH-07	(a) `CSTTT.TargetTheory.GenericHandoff.NativeComposition.PublicationReturn.native_full_temporal_composition` (b) `CSTTT.TargetTheory.GenericHandoff.NativeComposition.PublicationReturn.whole_signature_middle`	`Classical.choice, Quot.sound, propext`	(a) yes (b) yes

Continued on next page

Result / title	Registry	Headline Lean declarations	Axioms	In inv.
proposition 12 Current state is not always sufficient	TTN-04	(a) `CSTTT.POS04.ttn04_current_collision_memory` (b) `CSTTT.RouteTheory.noCurrentOnlyPredictor`	(a) `Classical.choice, Quot.sound, propext` (b) `none`	(a) yes (b) yes
theorem 4 Licences for dropping route memory	TTH-09	(a) `CSTTT.RouteLicenses.normalization_sufficient` (b) `CSTTT.RouteLicenses.flatness_sufficient` (c) `CSTTT.RouteLicenses.confluence_sufficient`	(a) `none` (b) `propext` (c) `none`	(a) yes (b) yes (c) yes
proposition 13 Holonomy without an arrow	TTN-03	(a) `CSTTT.RealControls.ttn03_exists`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 14 No canonical present	TTN-05	(a) `CSTTT.RealControls.ttn05_exists`	`Quot.sound, propext`	(a) yes
theorem 5 Continuation is relation-valued	TTH-08	(a) `CSTTT.TargetTheory.GenericHandoff.NativeComposition.NativeFission.native_relation_valued_temporal_fission` (b) `CSTTT.TargetTheory.GenericHandoff.NativeComposition.NativeFission.no_equivariant_selector`	`Classical.choice, Quot.sound, propext`	(a) yes (b) yes
proposition 15 Symmetry gives no ground for choosing	TTN-11	(a) `CSTTT.RealControls.ttn11_exists` (b) `CSTTT.RealControls.ttn11_no_invariant_unique_selector`	`Classical.choice, Quot.sound, propext`	(a) yes (b) yes
theorem 6 Ancestry underdetermines identity	TTF-07	(a) `CSTTT.FallbackTheory.StructuralAdjudication.actual_branch_identity_return`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 16 Branchwise preservation is not recombination	TTN-14	(a) `CSTTT.POS04.ttn14_marginals_do_not_fix_recombination`	`propext`	(a) yes
proposition 17 Independent twin	TTF-03	(a) `CSTTT.FallbackTheory.GuardedFusion.actual_independent_twin_exclusion`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 18 Reconstruction versus twin	TTF-06	(a) `CSTTT.FallbackTheory.StructuralAdjudication.destructive_reconstruction_return`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 19 Ancestry does not give realization	TTF-08	(a) `CSTTT.FallbackTheory.StructuralAdjudication.actual_package_insufficiency_return`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 20 Fusion under ancestry	TTF-04	(a) `CSTTT.FullNative02.ttf04_owned_operational_realization` (b) `CSTTT.FullNative02.every_main_output_warmup_supported` (c) `CSTTT.FullNative02.every_mature_output_fork_supported`	`Classical.choice, Quot.sound, propext`	(a) yes (b) yes (c) yes
theorem 7 Continuation by ancestry	TTF-05	(a) `CSTTT.FullNative02.IAPS.original_integrated_temporal_iaps` (b) `CSTTT.FullNative02.IAPS.both_child_requests_valid` (c) `CSTTT.FullNative02.IAPS.acquired_output_request_valid` (d) `CSTTT.FullNative02.IAPS.all_actual_outputs_three_lanes`	`Classical.choice, Quot.sound, propext`	(a) yes (b) yes (c) yes (d) yes
definition 5 Direct succession	TTD-01	(a) `CSTTT.TargetTheory.RawDirectSuccessorReturn.actual_validity_return`	`Classical.choice, Quot.sound, propext`	(a) yes

Continued on next page

Result / title	Registry	Headline Lean declarations	Axioms	In inv.
proposition 21 Validity of direct succession	TTD-01	(a) `CSTTT.TargetTheory.RawDirectSuccessor Return.actual_validity_return`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 22 What direct succession excludes	TTD-03	(a) `CSTTT.CorridorTheory.RawDirectSuccessor.actual_direct_exclusions` (b) `CSTTT.TargetTheory.RawDirectSuccessor Return.data_reachability_does_not_license_direct` (c) `CSTTT.TargetTheory.RawDirectSuccessor Return.literal_data_exclusion_false`	`Quot.sound, propext`	(a) yes (b) yes (c) yes
proposition 23 Two different modes	TTD-02	(a) `CSTTT.TargetTheory.RawDirectSuccessor Return.actual_modes_incomparable` (b) `CSTTT.CorridorTheory.RawDirectSeparation.overlap_not_direct`	`Quot.sound, propext`	(a) yes (b) yes
theorem 8 Unbounded maintenance on a finite factor	TTS-04	(a) `CSTTT.AutonomyTheory.RawRechargingPolicy.all_supported_streams_infinite_laws` (b) `CSTTT.AutonomyTheory.FiniteProgress.no_infinite_strict_finite_state_run`	(a) `Classical.choice, Quot.sound, propext` (b) `Quot.sound, propext`	(a) yes (b) yes
proposition 24 Exact defect semantics	TTS-02	(a) `CSTTT.RationalDefectTheory.exact_acceptance_iff` (b) `CSTTT.RationalDefectTheory.component_downward` (c) `CSTTT.RationalDefectTheory.scalar_does_not_determine_exact_acceptance` (d) `CSTTT.RationalDefectTheory.scalar_failure_with_same_schema`	(a) `Quot.sound, propext` (b) `Classical.choice, Quot.sound, propext` (c) `Classical.choice, Quot.sound, propext` (d) `Classical.choice, Quot.sound, propext`	(a) yes (b) yes (c) yes (d) yes
proposition 25 A bounded robustness box	TTS-03	(a) `CSTTT.RobustnessTheory.POS04Rational Margin.supported_box_event`	`Classical.choice, Quot.sound, propext`	(a) yes
proposition 26 Conditioning does not factor	(G26; no row)	(a) `CSTTT.FullNative.JointClaims.conditioned_joint_sum_of_products` (b) `CSTTT.FullNative.JointClaims.conditioning_factorization_countertrace`	`Classical.choice, Quot.sound, propext`	(a) yes (b) yes

Claims stated in prose

Section / claim	Registry	Headline Lean declarations	Axioms	In inv.
section 4.1 TTH-02	TTH-02	(a) `CSTTT.POS04.Assigned.stagedHeterogeneous PackageAssembly_generic`	`Classical.choice, Quot.sound, propext`	(a) yes
section 4.2 TTH-03	TTH-03	(a) `CSTTT.TransportTheory.bridge_frozen_snapshot_exists` (b) `CSTTT.TransportTheory.bundled HeterogeneousTemporalTransport_frozen`	`Classical.choice, Quot.sound, propext`	(a) yes (b) yes
section 4.2 TTH-10	TTH-10	(a) `CSTTT.RecordTheory.monotoneRecordDescent TransportsArrow_generic` (b) `CSTTT.RecordTheory.weakRecordMonotonicity Insufficient_generic`	(a) `Quot.sound, propext` (b) `propext`	(a) yes (b) yes

Continued on next page

Section / claim	Registry	Headline Lean declarations	Axioms	In inv.
section 8 TTF-01	TTF-01	(a) `CSTTT.FallbackTheory.RawPredictive Quotient.bridge_mk` (b) `CSTTT.FallbackTheory.RawPredictive Quotient.representative_independent` (c) `CSTTT.FallbackTheory.RawPredictive Quotient.unique_bridge` (d) `CSTTT.FallbackTheory.ReleasedPredictive Quotient.released_map_laws`	(a) `Classical.choice, Quot.sound, propext` (b) `Classical.choice, Quot.sound, propext` (c) `Classical.choice, Quot.sound, propext` (d) `Quot.sound, propext`	(a) yes (b) yes (c) yes (d) yes
section 8 TTF-02	TTF-02	(a) `CSTTT.FallbackTheory.GuardedFusion. actual_least_operational_ancestry`	`Classical.choice, Quot.sound, propext`	(a) yes
section 9 TTS-01	TTS-01	(a) `CSTTT.TargetTheory.RawDirectSuccessor Paths.actual_mixed_path_with_no_endpoint_ edge` (b) `CSTTT.CorridorTheory.RawDirectSuccessor. generalized_path_composition` (c) `CSTTT.CorridorTheory.ModelBoundPath.no_ closed_model_path`	(a) `Quot.sound, propext` (b) `Quot.sound, propext` (c) `Classical.choice, Quot.sound, propext`	(a) yes (b) yes (c) yes

D Repairs, and a supplementary result

D.1 Statements we repaired

While formalizing, we found several statements in our own earlier formulations to be false. We replaced each with a true statement and give both here, so that no reader mistakes the original for a result.

- *Licences for dropping route memory.* The original said that routes with the same boundary have equal endpoints *only if* a normalization, confluence or flatness licence holds. That is false: on a domain of three routes, two have equal endpoints and no licence holds. We prove only sufficiency (Theorem 4).
- *Direct succession and data.* The original said that direct succession excludes every data path. That is false: a genuine direct successor can coexist with an independent data path. We prove the four exclusions and the insufficiency of data reachability (Proposition 22).
- *Robustness.* The original said that every perturbation within the model's margin preserves every advance and every required condition. That is false. A correction by a quarter is not an integer difference. A receiver that outputs a constant keeps every number positive but fails the checks. We prove robustness in a rational extension, for one event and one box (Proposition 25).
- *Unbounded maintenance.* The original said that a policy on a finite invariant set yields an infinite run in a finite full state. That is impossible, since no infinite run that strictly progresses fits a finite full state. We prove it on a finite factor with unbounded registers (Theorem 8).
- *Reconstruction versus twin.* The original compared ancestry from one target to another and is refuted for every prefix. We use the rooted form (Proposition 18).
- *Identity.* The original made a claim about every identity relation, and the relation that identifies everything refutes it. We prove underdetermination for a stated class of identity relations (Theorem 6).

D.2 Conditioning does not factor

A natural shortcut for the joint behaviour of two branches is to condition on a shared event and multiply the conditioned marginals. That is wrong in general, and the fusion model's predecessor gives an exact case.

Proposition 26 (Conditioning does not factor)**.** *On the predecessor of the fusion model, a shared event has two outcomes, of weights* $\frac{1}{256}, \frac{9}{256}$ *(ratio* $1, 9$*). Conditioned on the event, and with the empty action word, the joint initial observation associated with the lower-weight outcome has probability* $\frac{1}{10}$*. Its two components are the corresponding initial observations of the two branches. Each has conditioned probability* $\frac{1}{10}$*, and their product is* $\frac{1}{100}$*. The correct joint law is a weighted sum of products of the branch laws over fixed worlds.*

E Reproducing the results

We designed every result to be rechecked from the repository. Lean 4.28.0 builds our development with its core library only and no external mathematics library. Our validator then checks each registered result against its declarations and confirms that only the standard axioms are used. Python scripts check the transcription of the finite models, export the result snapshot, and regenerate the evidence map. We give the commands, and the counts as of the release, in the artifact's reproducibility notes and in the data and code availability statement.

References

Samson Abramsky. Relational databases and Bell's theorem. In Val Tannen, Limsoon Wong, Leonid Libkin, Wenfei Fan, Wang-Chiew Tan, and Michael Fourman, editors, *In Search of Elegance in the Theory and Practice of Computation: Essays Dedicated to Peter Buneman*, Lecture Notes in Computer Science, pages 13–35. Springer, 2013. doi:10.1007/978-3-642-41660-6_2.

Samson Abramsky and Adam Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13(11):113036, 2011. doi:10.1088/1367-2630/13/11/113036.

Larissa Albantakis, Leonardo Barbosa, Graham Findlay, Matteo Grasso, Andrew M. Haun, William Marshall, William G. P. Mayner, Alireza Zaeemzadeh, Melanie Boly, Bjørn E. Juel, Shuntaro Sasai, Keiko Fujii, Isaac David, Jeremiah Hendren, Jonathan P. Lang, and Giulio Tononi. Integrated information theory (IIT) 4.0: Formulating the properties of phenomenal existence in physical terms. *PLOS Computational Biology*, 19(10):e1011465, 2023. doi:10.1371/journal.pcbi.1011465.

Bowen Alpern and Fred B. Schneider. Defining liveness. *Information Processing Letters*, 21(4): 181–185, 1985. doi:10.1016/0020-0190(85)90056-0.

Jean-Pierre Aubin. *Viability Theory*. Birkhäuser, Boston, 1991. ISBN 0-8176-3571-8.

Jean-Pierre Aubin, Alexandre M. Bayen, and Patrick Saint-Pierre. *Viability Theory: New Directions*. Springer, Berlin, Heidelberg, 2 edition, 2011. ISBN 978-3-642-16684-6. doi:10.1007/978-3-642-16684-6.

Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998. doi:10.1017/CBO9781139172752.

William A. Bauer. Against branching identity. *Philosophia*, 45(4):1709–1719, 2017. doi:10.1007/s11406-017-9870-8.

Catriel Beeri, Ronald Fagin, David Maier, and Mihalis Yannakakis. On the desirability of acyclic database schemes. *Journal of the ACM*, 30(3):479–513, 1983. doi:10.1145/2402.322389.

D. Bertsekas. Infinite time reachability of state-space regions by using feedback control. *IEEE Transactions on Automatic Control*, 17(5):604–613, 1972. doi:10.1109/TAC.1972.1100085.

F. Blanchini. Set invariance in control. *Automatica*, 35(11):1747–1767, 1999. doi:10.1016/S0005-1098(99)00113-2.

Michael A. Cerullo. Uploading and branching identity. *Minds and Machines*, 25(1):17–36, 2015. doi:10.1007/s11023-014-9352-8.

David J. Chalmers. Absent qualia, fading qualia, dancing qualia. In Thomas Metzinger, editor, *Conscious Experience*, pages 309–328. Schöningh/Imprint Academic, Paderborn, 1995. ISBN 0-907845-05-3.

David J. Chalmers. The singularity: A philosophical analysis. *Journal of Consciousness Studies*, 17 (9–10):7–65, 2010.

David J. Chalmers. The singularity: A reply. *Journal of Consciousness Studies*, 19:141–167, 2012.

Renzo Comolatti, Matteo Grasso, and Giulio Tononi. Why does time feel the way it does? Toward a principled account of temporal experience. *iScience*, 28(10):113434, 2025. doi:10.1016/j.isci.2025.113434.

Joseph Corabi and Susan Schneider. The metaphysics of uploading. *Journal of Consciousness Studies*, 19(7–8):26–44, 2012.

Barry Dainton. *Stream of Consciousness: Unity and Continuity in Conscious Experience*. Routledge, London, 2000. ISBN 0-415-22382-2.

Barry Dainton. *The Phenomenal Self*. Oxford University Press, Oxford, 2008. ISBN 978-0-19-928884-7. doi:10.1093/acprof:oso/9780199288847.001.0001.

Barry Dainton. Temporal consciousness. In *The Stanford Encyclopedia of Philosophy*. 2023. URL https://plato.stanford.edu/entries/consciousness-temporal/. First published 2010; substantive revision 17 March 2023.

Barry Dainton and Tim Bayne. Consciousness as a guide to personal persistence. *Australasian Journal of Philosophy*, 83(4):549–571, 2005. doi:10.1080/00048400500338856.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, Cham, 2021. doi:10.1007/978-3-030-79876-5_37.

Johan E. Gustafsson. Non-branching personal persistence. *Philosophical Studies*, 176(9):2307–2329, 2019. doi:10.1007/s11098-018-1127-2.

Katherine Hawley. Fission, fusion and intrinsic facts. *Philosophy and Phenomenological Research*, 71(3):602–621, 2005. doi:10.1111/j.1933-1592.2005.tb00473.x.

William James. *The Principles of Psychology*. Henry Holt, New York, 1890. Two volumes; chapter XV, "The perception of time".

John G. Kemeny and J. Laurie Snell. *Finite Markov Chains*. Undergraduate Texts in Mathematics. Springer-Verlag, New York, 1976. ISBN 0-387-90192-2. Reprint of the 1960 Van Nostrand edition.

Johannes Kleiner. Mathematical models of consciousness. *Entropy*, 22(6):609, 2020. doi:10.3390/e22060609.

Johannes Kleiner and Tim Ludwig. The case for neurons: a no-go theorem for consciousness on a chip. *Neuroscience of Consciousness*, 2024(1):niae037, 2024. doi:10.1093/nc/niae037.

Johannes Kleiner and Sean Tull. The mathematical structure of integrated information theory. *Frontiers in Applied Mathematics and Statistics*, 6:602973, 2021. doi:10.3389/fams.2020.602973.

Kim G. Larsen and Arne Skou. Bisimulation through probabilistic testing. *Information and Computation*, 94(1):1–28, 1991. doi:10.1016/0890-5401(91)90030-6.

Geoffrey Lee. Temporal experience and the temporal structure of experience. *Philosophers' Imprint*, 14(3), 2014. URL http://hdl.handle.net/2027/spo.3521354.0014.003.

David Lewis. Survival and identity. In Amélie Oksenberg Rorty, editor, *The Identities of Persons*, pages 17–40. University of California Press, Berkeley, 1976. doi:10.1525/9780520353060-002.

Michael L. Littman, Richard S. Sutton, and Satinder Singh. Predictive representations of state. In Thomas G. Dietterich, Suzanna Becker, and Zoubin Ghahramani, editors, *Advances in Neural Information Processing Systems 14*, pages 1555–1561. MIT Press, 2002.

René Mazala. Infinite games. In Erich Grädel, Wolfgang Thomas, and Thomas Wilke, editors, *Automata, Logics, and Infinite Games: A Guide to Current Research*, volume 2500 of *Lecture Notes in Computer Science*, pages 23–38. Springer, Berlin, Heidelberg, 2002. doi:10.1007/3-540-36387-4_2.

Robin Milner. *Communication and Concurrency*. Prentice Hall International Series in Computer Science. Prentice Hall, 1989. ISBN 0-13-114984-9.

Mikio Nakahara. *Geometry, Topology and Physics*. Institute of Physics Publishing, Bristol, 2 edition, 2003. ISBN 0-7503-0606-8. doi:10.1201/9781420056945.

M. H. A. Newman. On theories with a combinatorial definition of "equivalence". *Annals of Mathematics*, 43(2):223–243, 1942. doi:10.2307/1968867.

Robert Nozick. *Philosophical Explanations*. Belknap Press of Harvard University Press, Cambridge, MA, 1981. ISBN 0-674-66448-5.

Eric T. Olson. Personal identity. In *The Stanford Encyclopedia of Philosophy*. 2023. URL https://plato.stanford.edu/entries/identity-personal/. First published 2002; substantive revision 30 June 2023.

Derek Parfit. Personal identity. *The Philosophical Review*, 80(1):3–27, 1971. doi:10.2307/2184309.

Derek Parfit. *Reasons and Persons*. Oxford University Press, Oxford, 1984. ISBN 0-19-824615-3.

David Park. Concurrency and automata on infinite sequences. In *Theoretical Computer Science: 5th GI-Conference*, volume 104 of *Lecture Notes in Computer Science*, pages 167–183. Springer, 1981. doi:10.1007/BFb0017309.

John Perry. Can the self divide? *The Journal of Philosophy*, 69(16):463–488, 1972. doi:10.2307/2025324.

Ian Phillips. The temporal structure of experience. In Valtteri Arstila and Dan Lloyd, editors, *Subjective Time: The Philosophy, Psychology, and Neuroscience of Temporality*, pages 139–158. MIT Press, 2014. doi:10.7551/mitpress/8516.003.0012.

Hilary Putnam. Psychological predicates. In William H. Capitan and Daniel D. Merrill, editors, *Art, Mind, and Religion*. University of Pittsburgh Press, Pittsburgh, 1967. doi:10.2307/jj.6380610.6.

Cosma Rohilla Shalizi and James P. Crutchfield. Computational mechanics: Pattern and prediction, structure and simplicity. *Journal of Statistical Physics*, 104(3–4):817–879, 2001. doi:10.1023/A:1010388907793.

Sydney Shoemaker and Richard Swinburne. *Personal Identity*. Great Debates in Philosophy. Basil Blackwell, Oxford, 1984. ISBN 0-631-13208-2.

Ioannis Tsiokos. To throw a stone with six birds: On agents and agenthood, 2026a.

Ioannis Tsiokos. An audited closure architecture for consciousness: Exact finite models, structural theorems, and conditional realization, 2026b.

Ioannis Tsiokos. To flatten a stone with six birds: Critical pairs, holonomy, and confluence in rewriting systems, 2026c.

Ioannis Tsiokos. Six birds foundations iv: A catalog of layer-agnostic structural laws, 2026d.

Ioannis Tsiokos. Holonomy with memory in six birds theory: Predictive quotients, witnesses, and fixed-support route transport, 2026e.

Ioannis Tsiokos. Marking, erasure, and recombination on fixed support: Exact finite evidence from six birds theory, 2026f.

Ioannis Tsiokos. To notch a stone with six birds: Time as a theory artifact of order, measure, and arrow, 2026g.

Ioannis Tsiokos. Six birds protocol trap: Holonomy without entropy production, 2026h.

Ioannis Tsiokos. Auditing emergence in quantum error correction: A falsifiable instantiation of six birds theory, 2026i.

Ioannis Tsiokos. A closure architecture for consciousness: exact structure, consequences, and a conditional realization bridge, 2026j. Companion paper, in preparation.

Ioannis Tsiokos. When a public self-description cannot keep a system alive: exact forcing of a predictive self-coordinate, 2026k. Companion paper, in preparation.

Peter Unger. *Identity, Consciousness and Value*. Oxford University Press, 1990. ISBN 0-19-505401-6. doi:10.1093/oso/9780195054019.001.0001.

N. N. Vorob'ev. Consistent families of measures and their extensions. *Theory of Probability and Its Applications*, 7(2):147–163, 1962. doi:10.1137/1107014.

Clas Weber. The multiplicity objection against uploading optimism. *Synthese*, 205(6):226, 2025a. doi:10.1007/s11229-025-05057-9.

Clas Weber. Selective optimism about mind-uploading. *Journal of Consciousness Studies*, 32(1): 215–235, 2025b. doi:10.53765/20512201.32.1.220.

Bernard Williams. Personal identity and individuation. *Proceedings of the Aristotelian Society*, 57 (1):229–252, 1957. doi:10.1093/aristotelian/57.1.229.

Bernard Williams. The self and the future. *The Philosophical Review*, 79(2):161–180, 1970. doi:10.2307/2183946.